



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

Real Time Streaming Protocol: analisi e sicurezza. Un caso d'uso.

Relatore:

Prof. Angelo Furfaro

Dott. Francesco Blefari

Candidato:

Francesco Jiritano

Mat. 240259

ANNO ACCADEMICO 2024/2025

*A mamma, papà,
Alessia, Lilli e Zara*

Indice

Indice	2
1 Introduzione	5
1.1 Obiettivi della tesi	5
1.2 Struttura del documento	6
2 Real Time Streaming Protocol	8
2.1 Incarichi e Standard IETF	8
2.2 Protocollo TCP/IP	8
2.3 Funzionamento Generale di RTSP	9
2.4 Protocolli Correlati: RTP, RTCP, SDP	10
2.4.1 Real-Time Transport Protocol (RTP)	10
2.4.2 Real-Time Control Protocol (RTCP)	12
2.4.3 Session Description Protocol (SDP)	13
2.5 Meccanismi di Trasporto del Flusso Dati	13
2.5.1 Modalità RTP/AVP/UDP	14
2.5.2 Modalità RTP/AVP/TCP	14
2.6 Relazione e Confronto con HTTP	15
2.6.1 Codici di stato	16
2.6.2 Gestione dello Stato	18
2.6.3 Consegna dei Dati	18
2.6.4 Direzionalità delle Richieste	18
2.6.5 Gestione degli URI	19
2.6.6 Designazione dei Metodi di Controllo	20
3 Metodi del protocollo RTSP	22
3.1 Introduzione ai metodi	22

3.1.1	OPTIONS	22
3.1.2	DESCRIBE	23
3.1.3	SETUP	24
3.1.4	PLAY	25
3.1.5	PAUSE	26
3.1.6	TEARDOWN	27
3.1.7	ANNOUNCE	28
3.1.8	GET_PARAMETER	29
3.1.9	SET_PARAMETER	30
3.1.10	REDIRECT	31
3.1.11	RECORD	31
3.1.12	PLAY_NOTIFY	32
3.2	Stati della sessione RTSP	33
4	Sicurezza nel protocollo RTSP	37
4.1	Autenticazione in HTTP	37
4.1.1	Schema Basic	37
4.1.2	Schema Digest	39
4.1.3	Implementazione in RTSP	42
4.2	Sicurezza	44
4.2.1	Canale di controllo	44
4.2.2	Canale di flusso	44
4.2.3	AES	45
4.2.4	MIKEY	46
4.2.5	Crittografia in presenza di proxy	47
5	Realizzazione di un Server RTSP	50
5.1	Contesto dello Streaming Multimediale	50
5.1.1	Supporto di RTSP nelle applicazioni di streaming multimediale	50
5.1.2	Streaming on-demand	51
5.1.3	Streaming Real Time	52
5.2	Architettura delle componenti	52
5.2.1	Acquisizione del flusso video	53

5.2.2	L'elaborazione e la compressione del flusso	54
5.2.3	Distribuzione del servizio RTSP	55
5.2.4	Connessione del client	57
5.3	Conclusioni	58
6	Conclusioni	60

Capitolo 1

Introduzione

Negli ultimi decenni la trasmissione di dati multimediali in tempo reale è diventata essenziale, soprattutto in settori come la videosorveglianza, i live broadcast e le videoconferenze. Questa evoluzione ha posto nuove sfide, come la necessità di riuscire a gestire in modo efficiente flussi di dati tra il server e il client.

In questo scenario, i protocolli tradizionali basati su HTTP, che sono privi di vincoli temporali stringenti e di persistenza dello stato, non sono idonei per via del mancato controllo dinamico dei flussi.

Per colmare tali lacune, è stato sviluppato il Real Time Streaming Protocol (RTSP), che si afferma come uno standard importante poiché fornisce dei meccanismi di controllo remoto di sessioni multimediali. Il protocollo RTSP consente ad un client di poter interagire con un server tramite comandi come PLAY, PAUSE, RECORD e TEARDOWN.

Una peculiarità che differenzia RTSP da altre applicazioni per lo streaming è la sua capacità di separare il canale del controllo del flusso da quello di trasporto dei dati multimediali. La sua architettura modulare permette al protocollo di gestire in modo flessibile le sessioni, garantendo efficienza nel trattamento simultaneo di flussi audio e video.

1.1 Obiettivi della tesi

Lo scopo di questa tesi è analizzare in modo approfondito il funzionamento di Real Time Streaming Protocol (RTSP), con l'obiettivo di comprenderne la

struttura, le funzionalità e le potenzialità. In particolare, il lavoro si propone di:

- Descrivere l'architettura modulare del protocollo e il ruolo dei suoi principali componenti.
- Analizzare le modalità con cui RTSP interagisce e si integra con altri protocolli utilizzati nello streaming.
- Approfondire i metodi utilizzati nelle fasi di negoziazione e di trasmissione dei flussi multimediali.
- Esaminare gli aspetti legati alla sicurezza del protocollo.
- Presentare un caso d'uso pratico basato su un sistema end-to-end, mostrandone la configurazione, il funzionamento e il flusso.

1.2 Struttura del documento

Il documento è strutturato in sei capitoli, di seguito ne viene descritto il contenuto a partire dal secondo:

1. **Il capitolo 2** introduce e analizza in dettaglio il Real Time Streaming Protocol. Ne viene illustrata l'architettura e vengono esaminati i protocolli correlati come RTP e RTCP per il trasporto e il controllo del flusso di dati. Una sezione del capitolo è dedicata al confronto con HTTP, dove vengono evidenziate le differenze nel campo dello streaming.
2. **Il capitolo 3** prosegue l'analisi tecnica del protocollo concentrandosi sui suoi metodi, ovvero l'insieme dei comandi utilizzati per controllare e gestire la sessione di streaming. Nella parte finale del capitolo vengono descritti gli stati della sessione RTSP, illustrando come i metodi influenzino le transizioni di stato.
3. **Il capitolo 4** affronta il tema cruciale della sicurezza nel protocollo RTSP. La discussione inizia dall'analisi dei meccanismi di autenticazione in HTTP, esaminando in particolare gli schemi Basic e Digest. Successivamente, il capitolo si concentra sulle problematiche di sicurezza relative

ai canali di streaming, ovvero di controllo e di flusso. Nella sezione finale vengono illustrate le tecniche di crittografia in corrispondenza di server proxy, che permettono di garantire riservatezza e integrità dei dati.

4. **Il capitolo 5** introduce l'aspetto applicativo e la realizzazione di un server RTSP. Inizialmente viene inquadrato il protocollo nel contesto dello streaming multimediale, distinguendo l'uso di RTSP nello streaming on-demand e nello streaming Real Time. Il cuore del capitolo è dedicato alla descrizione dell'architettura delle componenti, dove vengono analizzate nel dettaglio le fasi di acquisizione ed elaborazione del flusso, la successiva distribuzione del servizio RTSP ed infine la connessione del client al server.
5. **Il capitolo 6** presenta le conclusioni del lavoro di tesi. In esso vengono riassunti i contenuti svolti, dall'analisi teorica del protocollo alla sua parte sperimentale con la realizzazione di un caso d'uso pratico. Il capitolo si conclude con l'indicazione dei possibili sviluppi futuri della ricerca, come l'integrazione della cifratura dei flussi.

Capitolo 2

Real Time Streaming Protocol

2.1 Incarichi e Standard IETF

Real Time Streaming Protocol (RTSP) nasce con lo scopo di fornire all'utente un insieme di funzionalità per il controllo da remoto di sessioni multimediali, consentendo ad un client di gestire la riproduzione di flussi di audio e di video attraverso dei comandi specifici.

Il protocollo è stato definito in modo ufficiale nel 1998 dall'Internet Engineering Task Force (IETF) all'interno del documento RFC 2326 [1]. In generale, le RFC (Request For Comments) sono una collezione ufficiale di documenti che riportano informazioni tecniche ed organizzative. In una RFC, infatti, si definiscono standard o protocolli che servono al normale funzionamento di Internet. Inoltre, nel 2016 è stata redatta una seconda versione del protocollo, che viene definita nel documento RFC 7826 [2]. Quest'ultimo offre una rivisitazione di RTSP, con il protocollo che viene reso più sicuro e completo.

2.2 Protocollo TCP/IP

L'architettura TCP/IP (Transmission Control Protocol/Internet Protocol) è un modello organizzato in quattro livelli, che dispone l'insieme di tutti i protocolli su cui si basa Internet.

Dal basso verso l'alto, in relazione alla Figura 2.1, vi sono i seguenti livelli:

- Network Interface Layer: è il livello di collegamento che si occupa di incapsulare e trasmettere i pacchetti di dati sull'hardware di rete.

- Internet Layer: è lo strato che gestisce la comunicazione tra le macchine. Ha la funzionalità di instradare i pacchetti verso la rete, identificando il percorso attraverso i router per raggiungere il destinatario dei dati.
- Transport Layer: è il livello che nel concreto gestisce la comunicazione tra due dispositivi, ovvero, nelle modalità TCP 2.5.2 o UDP 2.5.1, dopo che è stato deciso l'instradamento nell'Internet Layer.
- Application Layer: è lo strato che fornisce i protocolli che le applicazioni utilizzano per comunicare in rete, dunque, a questo livello viene gestita l'interattività tra l'utente e la macchina. Il Real Time Streaming Protocol, in particolare, si trova su questo strato, infatti viene definito un protocollo di controllo a livello applicativo.

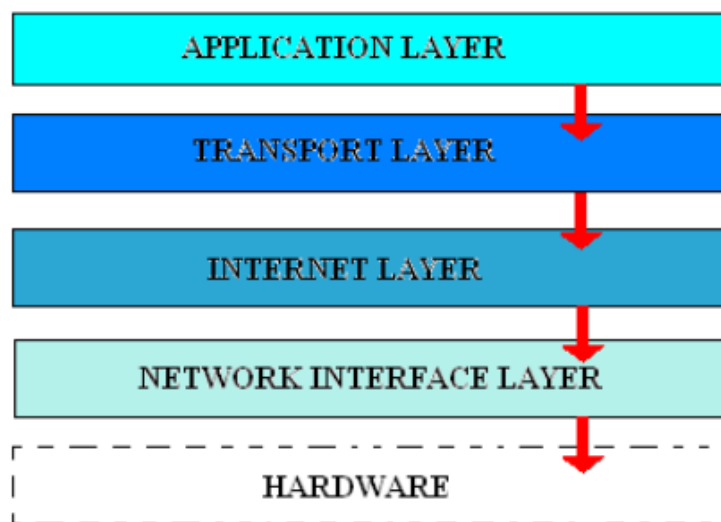


Figura 2.1: L'architettura TCP/IP [3]

2.3 Funzionamento Generale di RTSP

RTSP è un protocollo nato con lo scopo di occuparsi di stabilire e di supervisionare la trasmissione di dati multimediali in tempo reale su una rete IP (Internet Protocol). Il protocollo viene descritto come un "telecomando di rete" per il server multimediale, che assume tale definizione per via della sua funzione di gestione da remoto [1].

Un aspetto fondamentale del protocollo è l'assenza di un vero e proprio concetto di connessione RTSP, infatti il server mantiene una sessione, che è marcata da un identificatore. All'interno della sessione, un client può aprire e chiudere liberamente numerose connessioni TCP verso il server per inviare richieste, senza che la loro variazione interferisca con lo streaming continuo dei dati. L'importanza di RTSP risiede nella sua capacità di stabilire e di controllare più flussi multimediali sincronizzati e di interfacciarsi in modo efficace con protocolli di trasporto in tempo reale, come RTP 2.4.1, che è molto importante per la consegna end-to-end dei dati. Real Time Streaming Protocol sfrutta una struttura a moduli che consente una separazione netta tra il controllo delle sessioni e il trasporto dei contenuti, infatti si può avere una connessione dedicata a ciascuna funzionalità. Tale modularità del protocollo lo rende particolarmente adatto a situazioni in cui si richiedono flessibilità e interattività.

2.4 Protocolli Correlati: RTP, RTCP, SDP

Real Time Streaming Protocol non si preoccupa del trasporto dei flussi di dati multimediali, come audio e video, ma ha un compito di supervisione. Infatti, RTSP si occupa esclusivamente alla gestione dei flussi, stabilendo e controllando la loro consegna in tempo reale. Tuttavia, RTSP viene definito come un protocollo a livello applicativo che ha come compito quello di stabilire e supervisionare una o più sessioni di consegna, fornendo all'utente un framework estensibile per la trasmissione controllata e on-demand (su richiesta).

Per il reale trasferimento dei dati multimediali, RTSP utilizza il framework fornito dal Real-Time Transport Protocol (RTP), che si adotta in coppia con Real-Time Control Protocol (RTCP), il quale funge da protocollo di supporto.

2.4.1 Real-Time Transport Protocol (RTP)

Come riferito in [4], RTP è un protocollo di trasporto di rete che è stato ideato per fornire servizi di consegna end-to-end e gestire dati in tempo reale su reti unicast o multicast.

- La modalità di trasporto unicast rappresenta un meccanismo di consegna, dove i dati vengono indirizzati a un destinatario specifico. Il tra-

spporto unicast si usa solitamente per i servizi on-demand, con il client che fornisce al server un indirizzo di destinazione.

- Nella modalità di trasporto multicast, invece, i dati vengono trasmessi a più destinatari e viene utilizzata maggiormente per le presentazioni di media live (trasmissioni in diretta).

Trattando il protocollo di consegna RTP, bisogna innanzitutto definire uno stream multimediale, che è l'insieme di tutti i pacchetti RTP e RTCP generati da una singola sorgente all'interno di una sessione RTP. Il protocollo RTP ha bisogno che la totalità dei suoi stream media sia sincronizzata in base al loro timestamp, ovvero l'istante di campionamento scelto come punto di riferimento per la trasmissione.

RTP si usa convenzionalmente insieme al protocollo RTSP nei sistemi di streaming di dati multimediali, infatti il loro modo operativo è complementare. Il protocollo RTSP si occupa di stabilire connessione e di inviare i comandi, mentre RTP ha il compito di consegnare i pacchetti in tempo reale e in modo sincronizzato.

Per la consegna dei dati end-to-end in tempo reale, svolge un ruolo essenziale l'header del pacchetto RTP, che fornisce informazioni e metadati per ordinare il flusso che si sta ricevendo in base al numero di sequenza.

L'immagine mostrata nella Figura 2.2 raffigura la struttura dell'header di un pacchetto RTP, che presenta più righe di intestazione, ognuna costituita da 32 bit. Nel primo rigo sono presenti:

- Versione (V): identifica la versione del protocollo RTP che è stato impiegato.
- Padding (P): indica se sono stati aggiunti bit per questioni di allineamento.
- Estensione (X): specifica se è presente un'estensione rispetto all'header standard.
- Contributing Source Count (CC): stabilisce il numero di identificatori presenti nell'header.

- Marker (M): marca eventi significativi.
- Payload Type (PT): identifica il formato del payload, ovvero il tipo dei dati contenuti nel pacchetto.
- Sequence Number: specifica il numero del pacchetto, con la funzionalità di riuscire a ricostruire l'ordine corretto o per rilevare la perdita degli oggetti trasmessi. La particolarità che rende utile tale parametro è che esso viene incrementato di un valore pari ad uno per ogni pacchetto inviato, affinché quest'ultimo avrà un identificativo unico.

Nel secondo rigo è presente Timestamp, che ha una grande importanza per quanto riguarda lo streaming gestito da RTSP, poiché serve per la sincronizzazione del flusso. Infatti, il timestamp RTP ha lo scopo di indicare l'istante di generazione del pacchetto, quindi è utile per stabilire ordine e continuità.

Nel terzo rigo si trova Synchronization source (SSRC), utilizzato per identificare la sorgente di sincronizzazione. L'identificatore viene scelto in modo casuale, così da evitare che, all'interno di una stessa sessione RTP, due sorgenti condividano il medesimo identificatore SSRC.

Infine, nell'ultimo rigo è presente il Contributing Source (CSRC), che ha la funzionalità di produrre una lista di SSRC che contribuiscono al payload del pacchetto [5].

0-1	2	3	4-7	8	9-15	16-31
V	P	X	CC	M	PT	Sequence Number
Timestamp						
Synchronization source (SSRC) Identifier						
Contributing Source (CSRC)						

Figura 2.2: Formato dell'header RTP [6]

2.4.2 Real-Time Control Protocol (RTCP)

RTCP è un elemento essenziale per il normale funzionamento del protocollo RTP, infatti, quest'ultimo, per sua natura, non prevede garanzie sulla consegna

puntuale o sull'ordine di arrivo dei pacchetti. Su tale questione, il protocollo RTCP ricopre il ruolo principale di fornire feedback e monitoraggio sulla Quality of Service (QoS) erogato da RTP. RTCP si limita a fornire statistiche tecniche come il ritardo medio, la perdita dei pacchetti e la qualità del segnale, ma non è sufficiente a valutare la qualità percepita dall'utente [4].

2.4.3 Session Description Protocol (SDP)

Session Description Protocol, definito nell'RFC 2327 [7], ha lo scopo di descrivere le sessioni multimediali che sono disponibili ai client che ne fanno richiesta. Nel contesto di RTSP, SDP ricopre il ruolo di fornire le informazioni utili per poter partecipare alla sessione che è stata avviata. Session Description Protocol fornisce servizi essenziali come:

- L'enumerazione e la definizione dell'insieme dei flussi che costituiscono le presentazioni.
- La specificazione sui parametri di codifica richiesti per la decodifica del flusso multimediale [1].
- La descrizione dei parametri temporali come la durata complessiva della sessione o il Normal Play Time (NPT).

SDP ha una particolare importanza in RTSP poiché definisce i meccanismi di controllo, in particolare specifica se la sessione prevede l'aggregazione di più flussi.

Dunque, Session Description Protocol definisce la configurazione della sessione, alleggerendo il compito di RTSP, che si può concentrare sulla gestione dello stato e sulla sua attività di controllo e supervisione dei flussi.

2.5 Meccanismi di Trasporto del Flusso Dati

RTSP ha la funzione di controllare la consegna dei dati e fornisce i mezzi per scegliere i canali di consegna, che vengono negoziati tra client e server. Un utente ha la facoltà di proporre diverse opzioni di trasporto, tra cui UDP,

multicast UDP o TCP, con la negoziazione che avviene nell'header Transport della richiesta SETUP.

Il flusso Real Time Transport Protocol (RTP) segue un'architettura Client-Server che si utilizza in due modalità di trasmissione differenti, vale a dire RTP/AVP/UDP e RTP/AVP/TCP.

2.5.1 Modalità RTP/AVP/UDP

Nella modalità di trasmissione RTP/AVP/UDP, quando il client invia il comando SETUP per configurare la sessione RTSP, avviene la negoziazione con il server, con cui si decidono le porte UDP utili per la trasmissione del flusso RTP. Quando il server invia i pacchetti di audio e video, questi avranno timestamp e numeri di sequenza unici che servono per riordinarli e per controllare l'integrità del flusso. Con tale modalità, si stabiliscono, in generale, due connessioni di cui una per l'audio e l'altra per il video. Dunque, si avranno due connessioni RTP (per la trasmissione dei dati multimediali) e altrettante RTCP (per il monitoraggio del flusso).

Il protocollo UDP (User Datagram Protocol) rappresenta la configurazione più comune, poiché permette una trasmissione in tempo reale dei pacchetti senza connessione. Tale configurazione favorisce una bassa latenza poiché è priva di meccanismi di ritrasmissione, ma per questi motivi presenta una natura disordinata e inaffidabile. Tuttavia, questo meccanismo di trasporto è molto efficiente se viene monitorato in modo continuo, infatti, oltre alla comunicazione RTSP, è necessario supervisionare gli altri quattro canali, che sono di natura RTP e RTCP.

2.5.2 Modalità RTP/AVP/TCP

La configurazione RTP/AVP/UDP rappresenta la modalità di trasmissione che viene utilizzata di default, infatti RTP/AVP/TCP è utilizzata solamente quando vi sono vincoli stringenti riguardanti la sicurezza. Un evento che potrebbe portare all'adozione di TCP è l'esistenza di un firewall con richieste particolari. In generale, un firewall è un sistema di sicurezza della rete che ha funzionalità di limitare e, in alcuni casi, di bloccare il traffico in entrata o

in uscita all'interno di uno spazio di rete privato. Un'imposizione da parte di un firewall, nei confronti di un server, può essere quella di permettere l'invio dei metodi RTSP solamente all'interno della stessa connessione in cui viene trasmesso il flusso.

Dunque, in alternativa, viene adoperata la configurazione TCP (Transmission Control Protocol) che può avvenire in due modi diversi. Si parla di interleaving (interfogliamento) quando sulla stessa connessione TCP vengono inviati contemporaneamente il flusso multimediale, composto dai pacchetti RTP e RTCP, e quello di controllo RTSP. Tale pratica, però, è sconsigliata poiché potrebbe causare un overhead, ovvero un sovraccarico del sistema, per questo motivo si preferisce aprire delle connessioni TCP indipendenti. Con questa modalità i pacchetti RTP e RTCP saranno inviati su connessioni TCP distinte e separate dal canale di controllo in cui vengono inviati i metodi RTSP.

La Figura 2.3 mostra in modo schematico come il Real Time Streaming Protocol si posiziona in relazione alle applicazioni di streaming multimediale.

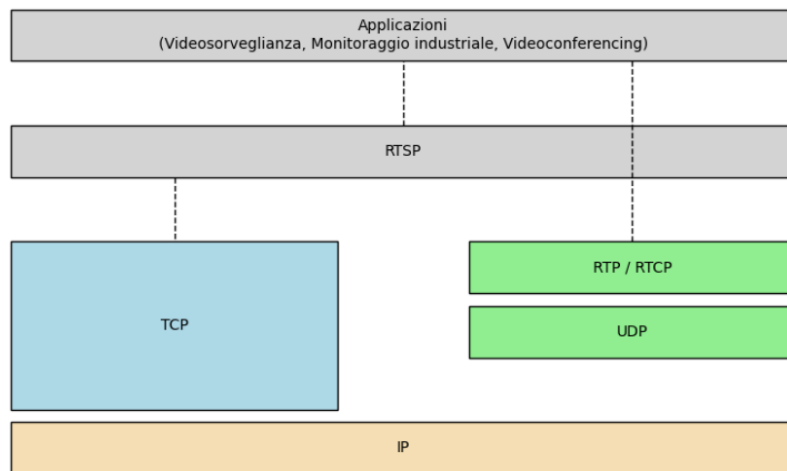


Figura 2.3: Architettura di RTSP [8]

2.6 Relazione e Confronto con HTTP

Il protocollo è simile nella sintassi e nel funzionamento al protocollo HTTP/1.1 (Hyper Text Transfer Protocol), poiché consente il riutilizzo dei meccanismi di estensione di HTTP, permettendo di usufruire dell'infrastruttura esistente in aree importanti come la sicurezza e la gestione dei proxy. Infatti RTSP

riutilizza i meccanismi di sicurezza web e di autenticazione, provenienti da HTTP, inclusi gli schemi Basic e Digest, che verranno trattati successivamente.

2.6.1 Codici di stato

La somiglianza tra questi due protocolli si può notare anche dall'utilizzo dei codici di stato, infatti, RTSP adotta lo stesso formato di riga di stato di HTTP. I codici di stato sono dei numeri interi composti da tre cifre che hanno lo scopo di indicare al client se la sua richiesta sia andata a buon fine o meno. La prima cifra del codice di stato specifica il tipo di risposta, mentre la seconda e la terza non hanno rilevanza nella loro classificazione. La prima cifra può assumere cinque valori diversi:

- 1xx (Informational): la richiesta è stata ricevuta e il processo sta continuando.
- 2xx (Success): l'azione è stata ricevuta, compresa e accettata con successo.
- 3xx (Redirection): si deve impiegare un'altra operazione per completare la richiesta.
- 4xx (Client Error): la richiesta contiene una sintassi con errori o che non può essere soddisfatta.
- 5xx (Server Error): il server ha fallito nel soddisfare una richiesta che all'apparenza sembra valida [1].

Nella Tabella 2.1 vengono mostrati alcuni dei codici di stato che ricorrono più spesso durante una tipica sessione RTSP, come è definita nello standard [1]. Nella prima colonna sono elencati i codici di stato, nella seconda i loro nomi, mentre nella terza i metodi da cui sono generati.

100	Continue	all
200	OK	all
201	Created	RECORD
250	Low on Storage Space	RECORD
300	Multiple Choices	all
305	Use Proxy	all
400	Bad Request	all
401	Unauthorized	all
404	Not Found	all
405	Method Not Allowed	all
406	Not Acceptable	all
407	Proxy Authentication Required	all
408	Request Timeout	all
415	Unsupported Media Type	all
451	Invalid parameter	SETUP
454	Session Not Found	all
455	Method Not Valid In This State	all
457	Invalid Range	PLAY
458	Parameter Is Read-Only	SET_PARAMETER
459	Aggregate Operation Not Allowed	all
460	Only Aggregate Operation Allowed	all
462	Destination Unreachable	all
500	Internal Server Error	all
501	Not Implemented	all
503	Service Unavailable	all
505	RTSP Version Not Supported	all
551	Option not supported	all

Tabella 2.1: Codici di stato RTSP

2.6.2 Gestione dello Stato

La differenza principale e più significativa tra i due protocolli risiede nel modello di gestione del loro stato.

- HTTP è definito come un protocollo stateless (senza stato), vale a dire che tratta ogni richiesta in modo indipendente dalle altre. Poiché privo di persistenza, lo rende inadatto per il controllo dinamico e continuo dei flussi multimediali, i quali richiedono la memorizzazione esplicita della posizione corrente e delle configurazioni di trasporto.
- RTSP, al contrario, è un protocollo stateful (con stato), dove il server deve mantenere uno stato stabilendo una sessione etichettata tramite l'header Session. Questo stato deve essere conosciuto dal server, che deve gestire le richieste successive relative a quel flusso specifico e deve avere cognizione del punto della presentazione in cui esso si trova.

2.6.3 Consegna dei Dati

- Nel modello HTTP la consegna dei dati è in-band, con i dati richiesti che vengono consegnati all'interno del corpo del messaggio di risposta. L'architettura HTTP offre alcuni vantaggi come la semplicità e la facilità di attraversamento dei firewall, poiché tutto il traffico avviene sulla stessa porta e sulla medesima connessione a livello di trasporto.
- In RTSP, invece, solitamente la consegna avviene out-of-band, vale a dire che il canale di trasporto dei media è diverso da quello di controllo del protocollo. Inoltre, in Real Time Streaming Protocol i dati vengono inviati tramite due protocolli separati: RTP e RTCP, con RTSP che fornisce il meccanismo per scegliere i canali di consegna attraverso l'header Transport.

2.6.4 Direzionalità delle Richieste

- HTTP è un protocollo asimmetrico di comunicazione, ovvero un modello in cui il client è l'unico che può emettere richieste. Con tale architettura

il server si deve limitare a fornire le risposte, senza poter indirizzare in modo autonomo altri flussi.

- RTSP, al contrario, adotta un modello di comunicazione bidirezionale, che è più adatto a gestire sessioni di streaming dinamiche e in tempo reale, le quali richiedono un'interazione continua con feedback immediati da parte del server, che sono indipendenti dalle azioni del client. Dunque, la direzionalità del protocollo permette ad entrambi gli agenti coinvolti, ovvero client e server, di poter inviare richieste per una gestione più flessibile dello stato favorendo lo sviluppo di sessioni più dinamiche e interattive.

2.6.5 Gestione degli URI

Una differenza fondamentale tra i due protocolli RTSP e HTTP risiede nel campo del Request-URI, con l'URI (Uniform Resource Identifier) che è una sequenza di caratteri che identifica in modo univoco e universale una risorsa.

- In RTSP la riga iniziale della richiesta (Request URI), che funge da identificatore della risorsa su cui deve essere applicato il metodo, deve contenere sempre l'URI assoluto (Absolute URI), cioè l'indirizzo completo del flusso multimediale o della risorsa che il client intende gestire o riprodurre. La scelta di imporre l'URI assoluto in ogni richiesta RTSP, come ad esempio SETUP o PLAY, è una decisione di design che garantisce autonomia e robustezza al protocollo di controllo. Di fatto, si assicura che ogni richiesta sia indirizzabile in modo immediato, senza vincoli sul fatto se tale risorsa sia accessibile o meno su internet.
- In HTTP, invece, si usa una struttura di Request-URI che segue tipicamente solo il percorso assoluto e non l'indirizzo completo, con le informazioni che sono trasmesse separatamente nell'header. Tale architettura facilita l'hosting virtuale, ovvero una pratica che consente a un singolo server fisico, dotato di un unico indirizzo IP, di ospitare diversi siti web o domini distinti. La separazione delle informazioni del percorso (path) dall'header permette al server di smistare in modo corretto le richieste verso la directory di documenti appropriata.

2.6.6 Designazione dei Metodi di Controllo

La differenza più significativa tra i metodi di RTSP e HTTP risiede nel loro scopo e nel loro impatto sullo stato del server e sulla gestione di flussi di dati continui. Mentre i metodi HTTP sono ottimizzati per il trasferimento o la manipolazione di entità, quelli RTSP sono progettati specificamente per il controllo continuo su una sessione multimediale in tempo reale.

Metodi di Recupero di Entità

- Il metodo HTTP GET è un meccanismo che ha il compito di recuperare la risorsa identificata dall'URI e di restituire il contenuto dell'entità stessa. Il metodo viene invocato da un client che vuole recuperare una determinata risorsa, con la risposta da parte del server che viene inviata sullo stesso canale.
- In RTSP, invece, esiste il metodo DESCRIBE, che sintatticamente è abbastanza simile a un GET per motivi legati all'estensione, ma si differenzia per la gestione dei dati. Il ruolo del metodo DESCRIBE si limita al recupero della descrizione della presentazione, che contiene le meta-informazioni necessarie per l'inizializzazione del media.

Metodi di Manipolazione e Controllo

Il metodo HTTP POST è utilizzato per inviare dati al server, spesso portando alla creazione o alla modifica di una risorsa. I metodi RTSP come SET_PARAMETER (o RECORD nella versione 1.0) possono influenzare i dati sul server, ad esempio avviando una registrazione o modificando le impostazioni di una sessione. Tuttavia, SET_PARAMETER ha uno scopo più specifico e granulare poiché si focalizza sull'amministrazione e sulla configurazione di una sessione RTSP già stabilita.

In sostanza, mentre l'efficacia dei metodi HTTP risiede nella loro semplicità e nella gestione di risorse in assenza di memoria, RTSP introduce metodi specifici (SETUP, PLAY, PAUSE, TEARDOWN) che non hanno riscontri diretti in HTTP, poiché sono progettati per manipolare esplicitamente gli stati (Init, Ready, Playing, Recording) di una sessione persistente. Questa differenza

riflette la diversa finalità dei due protocolli: in HTTP l'obiettivo primario è il trasferimento, in RTSP è il controllo temporale e l'allocazione di risorse di streaming out-of-band.

Nel capitolo successivo verranno mostrati in modo dettagliato i metodi di RTSP, soffermandosi sulle due versioni del protocollo.

Capitolo 3

Metodi del protocollo RTSP

3.1 Introduzione ai metodi

I metodi RTSP specificano l'operazione che deve essere eseguita sulla risorsa individuata dall'URI di richiesta e determinano il modo in cui client e server interagiscono per configurare, avviare, interrompere o modificare la riproduzione in streaming.

3.1.1 OPTIONS

Il metodo OPTIONS è un meccanismo fondamentale per la negoziazione delle capacità e per scoprire le funzionalità supportate dal server. Il suo scopo principale è quello di recuperare le opzioni di comunicazione messe a disposizione dall'agente che riceve la richiesta.

Con la seconda versione di RTSP [2], il metodo acquisisce una natura bidirezionale, per cui la richiesta può essere emessa da un client verso un server, ma anche viceversa. Il metodo OPTIONS può essere emesso in qualsiasi momento, anche perché non influenza lo stato. Nel caso in cui un client dovesse inviare una richiesta standard che non modifica lo stato della sessione, potrebbe, però, contribuire a prolungarne la durata.

La risposta al metodo OPTIONS è fondamentale per comunicare le funzionalità supportate dall'agente RTSP. La sua struttura è vincolata da requisiti precisi che riguardano gli header:

1. Header Public (Obbligatorio): deve sempre essere incluso nella risposta a una richiesta OPTIONS, indipendentemente dall'ambito definito dal Request-URI. Il suo scopo è elencare in modo esaustivo l'insieme dei metodi RTSP che sono supportati dall'agente rispondente (server o proxy).
2. Header Allow (Condizionale): deve essere incluso nella risposta solo nel caso in cui l'ambito della richiesta sia limitato a una specifica risorsa multimediale. Tale header ha la funzione di enumerare l'insieme dei metodi che sono consentiti per quella specifica risorsa, differenziandoli, se necessario, dai metodi supportati in generale (elencati in Public). La sua presenza non è richiesta se l'insieme dei metodi autorizzati per la risorsa coincide perfettamente con l'insieme elencato nell'header Public.
3. Header Supported (Opzionale nella Richiesta): può essere incluso nella richiesta OPTIONS al fine di interrogare le funzionalità estese supportate dall'agente RTSP che riceve la richiesta.

Gestione degli Errori

Se la risorsa specificata nel Request-URI non risulta disponibile o accessibile, l'agente RTSP dovrebbe restituire un codice di risposta appropriato, tipicamente appartenente alle classi 3xx (Reindirizzamento) o 4xx (Errore Client).

3.1.2 DESCRIBE

Il metodo DESCRIBE risulta di fondamentale importanza per recuperare dal server la descrizione di una presentazione o di un oggetto multimediale identificato dall'URI della richiesta. Senza tale metodo, sarebbe impossibile per il client acquisire le informazioni necessarie per la successiva riproduzione.

L'utilizzo di DESCRIBE prevede che il client invii una richiesta indirizzata alla risorsa interessata, includendo l'header Accept, impiegato per specificare i formati di descrizione che il client è in grado di comprendere e accettare. Successivamente, se la richiesta può essere soddisfatta, il server deve risponde-

re, fornendo una descrizione della risorsa richiesta dal client, che deve essere inserita nel corpo del messaggio di risposta.

L'uso del metodo ha un ruolo centrale nella fase di inizializzazione dei media di una sessione RTSP. La risposta DESCRIBE deve contenere tutte le informazioni necessarie per l'avvio dei media relativi alle risorse descritte, infatti presenta elementi cruciali come parametri, indirizzi di rete e set di codifiche. È importante tenere presente che l'inizializzazione dei media non è necessariamente vincolata al metodo DESCRIBE. In alternativa, le informazioni possono essere ottenute tramite altri protocolli o attraverso l'interfaccia utente a righe di comando.

3.1.3 SETUP

Il metodo SETUP permette a un client di specificare il meccanismo di trasporto che intende adoperare per la trasmissione del flusso di audio e video in streaming. Una volta definito il trasporto, è possibile instaurare una sessione RTSP sul server, che è essenziale per il funzionamento del protocollo, poiché quest'ultimo ha bisogno di mantenere lo stato della sessione.

Quando un client invoca il metodo, si impone al server di allocare le risorse necessarie per il flusso di dati e avviare la sessione RTSP. Una volta ricevuta la richiesta, il server deve generare e restituire un identificatore di sessione, stabilendo così una correlazione con le eventuali richieste successive.

Il metodo è essenziale per la negoziazione dei parametri di trasmissione, la quale viene gestita con l'header Transport. In esso il client specifica i parametri desiderati, come ad esempio i protocolli di trasporto, le porte e gli indirizzi di origine o destinazione. Ricevuta la richiesta, il server invia la risposta includendo il medesimo header ed elencando i parametri che sono stati utilizzati.

RTSP consente ad un client di inviare una richiesta SETUP per una nuova risorsa includendo un identificatore di sessione già esistente. Il server integra la nuova risorsa nella sessione corrente, se tale operazione non è consentita, allora deve rispondere con l'errore 459 Aggregate Operation Not Allowed.

Inoltre, con la nuova versione di RTSP, è possibile emettere un comando SETUP per uno stream già avviato con lo scopo di modificare i parametri di

trasporto, come ad esempio il cambio da TCP a UDP, per permettere di ottimizzare il trasporto.

3.1.4 PLAY

Il metodo PLAY è essenziale per il controllo della sessione, infatti è il comando che attiva l'erogazione effettiva di streaming. La funzione principale del metodo è indicare al server di cominciare a trasmettere i dati tramite il meccanismo di trasporto stabilito nel SETUP.

Il client può emettere una richiesta di PLAY solo se la sessione RTSP si trova negli stati Ready o Pause. Con il comando PLAY che deve includere l'intestazione Session per indicare in modo inequivocabile al server chi stia tentando di avviare il flusso dati, garantendo così integrità e continuità.

Inizialmente, nella versione originaria del protocollo, le richieste venivano inserite in una coda, seguendo una sequenza temporale precisa. Successivamente, con l'introduzione di RTSP 2.0, se il server riceve una nuova richiesta PLAY mentre lo stream si trova già nello stato Play, la richiesta corrente viene sostituita immediatamente con quella appena ottenuta.

Nella descrizione del metodo PLAY, è cruciale la trattazione del Normal Play Time (NPT), che rappresenta la posizione assoluta del flusso a partire dall'inizio della presentazione. L'NPT avanza normalmente quando la riproduzione procede alla velocità standard, progredisce più rapidamente durante la modalità di avanzamento veloce, diminuisce in caso di scansione inversa (fattore di scala negativo) e rimane invariato durante la pausa. L'inizio di una presentazione corrisponde a 0.0 secondi, con i valori negativi che intuitivamente non sono definiti. Inoltre, viene stabilita la costante "now" che indica l'istante corrente di un evento live e che non è impiegato per contenuti on-demand.

Il client nel momento in cui invia la richiesta può scegliere se specificare o meno l'intervallo temporale della presentazione da cui il flusso deve partire, se viene omesso, implica che venga ripresa dal pause point, ovvero dal punto di pausa corrente. Il server, una volta ricevuta la richiesta, verifica l'esistenza della sessione e se l'operazione viene accettata, deve indicare nella risposta l'intervallo temporale della riproduzione.

Inoltre, il server fornisce alcuni metadati di sincronizzazione come il numero di sequenza e il timestamp RTP iniziale, che sono importanti per il normale funzionamento del Normal Play Time, assicurandosi che l'applicazione a livello superiore possa decodificare con precisione i dati multimediali.

Il timestamp è importante per la mappatura del tempo di trasporto di NPT e per la sincronizzazione dei diversi flussi multimediali, in particolare è cruciale per la gestione della pausa.

Il numero di sequenza, invece, serve principalmente a garantire l'integrità del flusso, considerando che vi sono operazioni di navigazione, come il seek, che possono provocare discontinuità o duplicazioni. In generale, il seek è una richiesta di riproduzione a partire da un punto specifico nella linea temporale del contenuto.

Per evitare problemi di discontinuità o di duplicazioni, è essenziale che sia il numero di sequenza che il timestamp rimangano coerenti attraverso i salti di NPT. Questo garantisce integrità e ordine logico a livello di pacchetto RTP, permettendo al client di riprendere l'elaborazione del flusso senza errori dovuti al cambiamento del punto temporale.

3.1.5 PAUSE

Il metodo PAUSE rappresenta un requisito fondamentale per la gestione interattiva e per la riproduzione su richiesta (on-demand) dei flussi multimediali controllati dal Real Time Streaming Protocol. La funzione principale del metodo è l'interruzione immediata della consegna del flusso multimediale. Infatti provoca la sospensione della distribuzione dello streaming in modo temporaneo, per un certo range. In seguito a una richiesta PAUSE, inviata da un client che fornisce l'URI specifico per la sessione, viene arrestata la consegna dei dati e successivamente il server conserva le risorse associate alla sessione. Il server, comunque, ha la scelta di poter terminare la sessione e di liberare queste risorse se la pausa supera il valore del timeout stabilito nell'header Session durante la fase di SETUP.

Il server, per consentire una ripresa precisa della riproduzione, in seguito a una pausa, fornisce al client il punto esatto in cui la riproduzione è stata sospesa, rispondendo con il messaggio "200 OK" e includendo l'header Range, che indica

il punto di pausa e anche la porzione di flusso che deve ancora essere riprodotta, relativamente all'intervallo della richiesta Play, precedentemente inviata.

Il metodo può essere applicato sia a un singolo flusso che a un gruppo di presentazioni, con il comando PAUSE che deve essere inviato utilizzando l'URI di controllo aggregato, con l'effetto di interrompere tutte le stream relative a quella sessione attiva. La peculiarità è che sia impossibile poter sospendere un singolo flusso all'interno di una sessione aggregata, infatti a tale richiesta, il server risponderà con il codice di errore "460 Only Aggregate Operation Allowed".

3.1.6 TEARDOWN

Il metodo TEARDOWN è indispensabile per la gestione dello stato della sessione, infatti, la sua funzione primaria è l'interruzione del flusso di dati e il rilascio delle risorse precedentemente allocate dal server per uno specifico stream. TEARDOWN differisce da PAUSE, poiché il secondo sospende il flusso, in modo temporaneo, senza liberare le risorse trattenute dal server e mantiene invariata la sessione, mentre il primo metodo ne determina la conclusione in modo definitivo.

Il client, dopo aver inviato una richiesta di TEARDOWN per la chiusura della sessione di streaming, deve includere obbligatoriamente l'header Session che serve a identificare in modo univoco il flusso o la connessione specifica che deve essere terminata.

Un TEARDOWN che utilizza un URI multimediale all'interno di una sessione aggregata può essere eseguito solo se essa si trova nello stato Ready. Tale richiesta TEARDOWN rimuove solo il flusso multimediale indicato e le risorse associate dalla sessione. Di conseguenza, la sessione potrebbe tornare a un controllo non aggregato, nel caso in cui, dopo l'operazione, rimanga associato un unico flusso.

Con la nuova versione del protocollo, diversamente dalla prima, è permesso al server di inviare richieste di TEARDOWN al client per segnalare l'impossibilità di continuare nella riproduzione, poiché stato costretto ad interrompere la sessione in corso. Le ragioni per cui un server invii una richiesta TEARDOWN possono essere differenti, come l'inattività prolungata del client o la

manutenzione del server. In questo contesto dal server al client la richiesta TEARDOWN deve includere l'header TERMINATE-REASON per specificare il motivo della chiusura. Inoltre, il server può invocare il comando TEARDOWN con l'URI wildcard (*), senza includere un header Session, che indica a tutte le sessioni e richieste in sospeso che devono essere terminate.

Relativamente al metodo TEARDOWN, è importante trattare lo studio dei proxy RTSP, ovvero dei server che fungono da intermediari per le richieste inviate dai client agli altri server e svolgono funzioni essenziali come il caching, la traduzione di indirizzi, l'accesso e la sicurezza.

Al seguito di un comando TEARDOWN, da parte di un server che utilizza l'URI wildcard, tutti i proxy che sono intervenuti nel percorso dovrebbero eseguire le seguenti operazioni:

1. Rispondere alla richiesta TEARDOWN ricevuta dal server.
2. Disconnettere il canale di controllo dal server che lo richiede.
3. inoltrare la richiesta TEARDOWN a ciascun client con una sessione attiva o una richiesta senza risposta.

Bisogna inoltre considerare che il proxy è responsabile dell'accettazione delle risposte TEARDOWN che provengono dai propri client e non devono essere inoltrate al server originale.

Il proxy si assume la totale responsabilità in caso di chiusura forzata del server o di fallimento, della gestione della sessione, fungendo da garante per la robustezza del sistema. Di fatto, il proxy si occupa di ripulire il proprio stato interno e di notificare ai client il comando TEARDOWN, così da permettere la deallocazione delle risorse di rete locali, come ad esempio le connessioni TCP impiegate nel protocollo RTSP.

3.1.7 ANNOUNCE

Il metodo ANNOUNCE è stato implementato con la caratteristica di poter svolgere due compiti diversi, ovvero pubblicare la descrizione e aggiornarla in tempo reale. ANNOUNCE, quando al contrario viene trasmesso dal client in direzione del server, ha la funzionalità di pubblicare la descrizione di una

presentazione o di un oggetto multimediale che è stato identificato dall'URI. Il metodo, quando, al contrario, viene inviato dal server verso il client, si usa per aggiornare la descrizione della sessione in tempo reale, favorendo l'efficienza delle presentazioni live. Quando si aggiunge un nuovo flusso multimediale a una presentazione attiva, invece di inviare solamente la descrizione dei nuovi componenti, viene ritrasmessa per intero.

Nella seconda versione di RTSP il metodo ANNOUNCE è stato deprecato dal protocollo base, ovvero è stato considerato obsoleto, in quanto è stato ufficialmente sostituito. Quindi se per RFC 2326, che è relativa alla prima versione del protocollo, il metodo era ritenuto importante per la gestione in tempo reale, nella successiva RFC 7826, che trattava RTSP 2.0, la sua funzionalità è stata rimossa.

3.1.8 GET_PARAMETER

Nella prima versione del protocollo, il metodo GET_PARAMETER era stato definito per recuperare il valore di un parametro specifico relativo a una presentazione o ad un oggetto multimediale identificato dall'URI di richiesta. Inoltre, il metodo aveva una sua particolarità, infatti, se fosse stata inviata una richiesta senza inserire un corpo dell'entità, si sarebbe potuto testare la vitalità del client o del server. Con l'introduzione della seconda versione del protocollo, il metodo è stato modificato per renderlo più versatile e più utile. Il client può richiedere informazioni al server in due modi diversi: tramite header o attraverso il corpo del messaggio, con RTSP, che per prevenire fallimenti, è stato irrobustito per poter gestire errori specifici.

Il protocollo contiene dei meccanismi di controllo e di segnalazione degli errori per salvaguardare la comunicazione di fronte a situazioni poco standard. Quando il client invia nel corpo del messaggio dei parametri che non sono stati compresi dal server, esso risponde inviando il codice di stato 451 (Parameter Not Understood). Inoltre, la risposta deve contenere nel corpo del messaggio i parametri che il server non ha compreso, così che il client possa prendere atto delle informazioni che non sono state percepite, a causa di un invio errato di parametri da parte del client.

Un altro esempio di errore che viene gestito da RTSP è quando il media type, ovvero la natura e il formato del contenuto multimediale, che è stato adottato per trasmettere i parametri nel corpo del messaggio non è stato supportato. A tale richiesta, il server risponde al client con il codice di errore 415 (Unsupported Media Type).

Il metodo GET_PARAMETER è utile anche per il mantenimento in vita (keep-alive) della sessione RTSP. Infatti, il client quando invia la richiesta e questa viene accettata dal server, il timer della scadenza viene ripristinato o prolungato.

3.1.9 SET_PARAMETER

Il metodo SET_PARAMETER ricopre un ruolo importante nella gestione dello stato della sessione, infatti consiste nel richiedere al server le impostazioni di nuovi valori per i parametri desiderati relativi a una specifica presentazione o singolo flusso multimediale, che viene identificato dall'URI della richiesta. Il metodo, veicolando informazioni per la configurazione di alcuni parametri, avrà una richiesta accompagnata da un corpo del messaggio con le modifiche che si intendono eseguire, inserendole nell'header Content-Type. Come per GET_PARAMETER, anche per il metodo SET_PARAMETER, RTSP fornisce delle eccezioni ed una gestione degli errori abbastanza efficiente.

Quando si invia un comando SET_PARAMETER composto da più valori, questo viene accettato dal server solamente se tutti i parametri sono permessi, se uno è irregolare, allora la richiesta viene rifiutata. In tal caso, se il destinatario non supporta il media type, questo deve rispondere con il codice di errore 415 (Unsupported Media Type). Al client, quando deve inviare una richiesta SET_PARAMETER è raccomandato che una richiesta contenga un singolo parametro, infatti, in caso di fallimento, è più semplice comprendere la causa che ha fatto sorgere l'errore.

Un altro codice di errore che protegge la comunicazione dagli errori provocati dal metodo SET_PARAMETER è 458 (Parameter Is Read-Only), che indica che si sta cercando di modificare un parametro che si può consultare, ma non è stato configurato per essere variato.

3.1.10 REDIRECT

Il metodo REDIRECT è una funzione asimmetrica del protocollo, infatti può essere invocato solo dal server verso il client e garantisce flessibilità e continuità del servizio multimediale in caso di possibili manutenzioni. Il metodo REDIRECT ha come funzione primaria quella di inviare informazioni al client riguardo il servizio che sta ricevendo, riferendo che esso verrà terminato. Inoltre, il metodo deve anche indicare la posizione del server alternativa a cui il client deve reindirizzare le sue richieste future. Il reindirizzamento può essere utile per diversi motivi, come, per esempio, il bilanciamento del carico (Load Balancing), ovvero la ridistribuzione delle richieste verso un server meno congestionato o che si trova più vicino al client. Dunque, questo metodo si pone a risolvere i problemi legati alla scalabilità del server, diminuendo in modo efficiente i tempi di attesa per i client.

Quando un client riceve una richiesta di REDIRECT deve inviare due comandi: TEARDOWN e SETUP. Il comando di TEARDOWN serve ad interrompere il flusso di dati e a rilasciare le risorse che in quel momento erano utilizzate, in questo modo si libera spazio essenziale per evitare la congestione. Considerando la rete internet, non si può sottovalutare il fenomeno della congestione, ovvero il sovraccarico della rete e l'esaurimento delle risorse applicative. Nel protocollo Real Time Streaming Protocol la congestione si può notare attraverso RTCP, ovvero il protocollo che controlla l'analisi della trasmissione dei pacchetti effettuata da RTP, che tiene traccia delle perdite dei pacchetti dovuti a una congestione persistente.

Il client, dopo aver inviato il comando di TEARDOWN, per chiudere la connessione precedente, deve inoltrare una richiesta di SETUP, creando una nuova sessione e inserendo nel campo Request-URI l'URI assoluto specificato nell'header Location trasmesso dal server tramite REDIRECT.

3.1.11 RECORD

Il metodo RECORD è importante nel protocollo RTSP poiché permette di eseguire l'operazione di registrazione dei flussi multimediali. RECORD è stato implementato con lo scopo di iniziare la registrazione di un intervallo temporale

specifico, che è stato desiderato dal client. Dal punto di vista temporale, il metodo RECORD è molto stringente, infatti un singolo errore di gestione potrebbe non portare a termine, in modo corretto, la risoluzione della richiesta inviata dal client. Per una migliore usabilità ed affidabilità, viene incluso un timestamp nella richiesta che deve indicare l'orario di inizio e di fine della registrazione desiderata dal client. Nel caso in cui il client non fornisca un intervallo temporale, si utilizza, per la registrazione, la coppia di orari di inizio e fine della presentazione, che viene specificata nella sua descrizione.

Il server decide se memorizzare i dati registrati sotto l'URI della richiesta o un altro URI. Se il server non utilizzasse l'URI della richiesta, la risposta dovrebbe essere 201 (Created) e contenere un'entità che descrive lo stato della richiesta e fa riferimento alla nuova risorsa, e un'intestazione Location.

Successivamente, nella seconda versione del protocollo, RECORD è stato deprecato e tutte le sue funzionalità sono state rimosse, con la registrazione che non è gestita da un metodo definito. Per ovviare a tale rimozione del metodo, con RTSP 2.0 la registrazione avviene in modo diverso, infatti il server riferisce al client quando può offrire servizi di registrazione live. Ricevuta la notifica proveniente dal server, il client può accedere in modo casuale ai contenuti multimediali da quel momento fino alla fine della sessione e inoltre, può richiedere la riproduzione in qualsiasi punto all'interno della finestra in cui si trova.

3.1.12 PLAY_NOTIFY

Il metodo PLAY_NOTIFY originariamente non era presente nella prima versione del protocollo, ma è stato introdotto in RTSP 2.0 per svolgere alcune mansioni che precedentemente non erano presenti. PLAY_NOTIFY è un metodo che viene inviato dal server nei confronti del client e ha il compito di informare l'utente riguardo gli eventi asincroni che si verificano all'interno di una sessione che in quel momento si trova nello stato di Play. La funzionalità del metodo è permessa dalla struttura di RTSP, infatti si basa sull'esistenza di una connessione persistente tra il server e il client, ciò, quindi, permette la possibilità di ricezione delle richieste da parte dell'utente. Quando un client non dovesse capire la ragione della notifica, deve rispondere con il codice di errore 465 (Notification Reason Unknown).

RFC 7826, ovvero lo standard per la seconda versione del protocollo RTSP, definisce come i server possono inviare PLAY_NOTIFY:

- End-of-stream (fine del flusso), che indica il completamento vicino o effettivo della richiesta di PLAY, indicando, dunque, la fine della consegna del flusso multimediale.
- Media-Properties-Update (Aggiornamento delle Proprietà dei Media), che può specificare che proprietà dei media o il loro intervallo disponibile per la riproduzione sono stati aggiornati.
- Scale-Change (Modifica della Scala di riproduzione), che viene inviata dal server quando questo è costretto a modificare la sua scala, ovvero la velocità di riproduzione, che precedentemente è stata richiesta dal client.

3.2 Stati della sessione RTSP

La sessione RTSP, per via della sua natura persistente, che permette di mantenere lo stato, funge da contesto comune su cui agire tra il client e il server. In generale, il client chiede al server di creare una sessione RTSP che deve essere mantenuta fino al termine della comunicazione. Le macchine a stati descrivono il comportamento del protocollo dall'inizializzazione della sessione RTSP fino al momento della sua terminazione.

Gli stati principali assunti dal server RTSP sono: Init, Ready, Playing e Recording.

Nello stato iniziale Init la sessione non esiste, deve essere ancora creata, infatti non è attiva nessuna riproduzione nel server a favore del client. Il server, in quel momento, non ha ancora allocato le risorse di memoria, come indirizzi o porte, inoltre, il client non possiede un ID di sessione univoco a cui accedere. Il client non ha inviato il comando di SETUP, quindi non sono ancora stati negoziati il protocollo o il meccanismo di trasferimento che, in seguito, verranno utilizzati nel corso della sessione. In questo stato il client può inviare solamente richieste che non hanno bisogno che ci sia una sessione attiva, come OPTIONS o DESCRIBE.

Partendo da Init, quando il client invia una richiesta SETUP e questa viene accettata, la sessione passa allo stato Ready, con il server che alloca le risorse e fornisce come risposta il Session ID. Nello stato Ready, dunque, la sessione è stata stabilita, avendo concordato con il client il meccanismo di trasporto dei dati multimediali. La sessione nello stato Ready è pronta per avviare la riproduzione, infatti si tratta di uno stato intermedio e di attesa, in cui si configura un momento di equilibrio che si può raggiungere anche dopo un comando PAUSE inviato nello stato Playing.

In generale, per transitare nello stato di Playing, bisogna inviare un comando PLAY a partire da Ready. Nello stato Playing il server invia i dati multimediali al client attraverso l'uso del canale di trasporto che in precedenza era stato negoziato con il comando SETUP. Contemporaneamente all'invio dei pacchetti multimediali RTP, il server monitora la qualità del servizio tramite il protocollo RTCP, regolando il flusso di dati se la situazione ne necessita. Quando ci si trova in Playing e si riceve un comando di TEARDOWN da parte del client, il server deve liberare le risorse associate a quella sessione, la quale cessa di esistere.

A partire da Playing, quando il client invia il comando RECORD, la sessione entra nello stato di Recording e viene avviata la registrazione di un intervallo temporale che è stato deciso dall'utente nella descrizione della presentazione. Recording è uno stato in cui il flusso si inverte, infatti è il client a inviare i dati verso il server, in modo inverso alla direzione tradizionale.

Il protocollo RTSP è stato progettato supportando l'invito all'interno di una conferenza da parte di un server, che ha il compito di registrare i media di una presentazione.

Si può uscire dallo stato di Recording attraverso due comandi: PAUSE e TEARDOWN. Quando un client invia una richiesta PAUSE, quando la sessione si trova nello stato di Recording, il server interrompe la registrazione, e si entra nello stato di Playing. Infine, se un client invoca il comando TEARDOWN, come già visto prima, la sessione cessa di esistere.

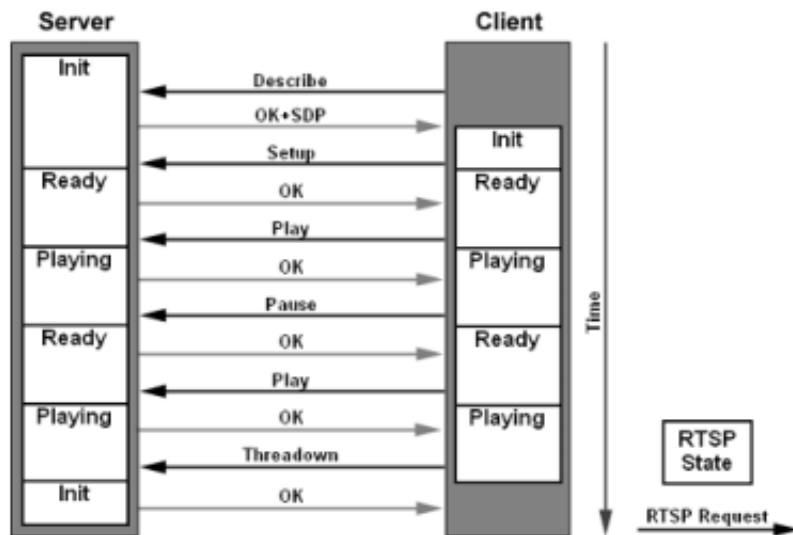


Figura 3.1: Un esempio di una sessione RTSP tra client server [9]

Nell'immagine che mostra la Figura 3.1 si può prendere visione di un esempio di sessione Real Time Streaming Protocol, dove un client invia al server una richiesta DESCRIBE per chiedere delle informazioni. Il server, che si trova in uno stato Init, ovvero che non ha ancora ricevuto il comando di SETUP, risponde al client con un messaggio di conferma OK seguito dal SDP (Session Description Protocol).

Successivamente, il client invia il comando di SETUP che provoca la creazione della sessione, con il server che alloca le risorse per il flusso e stabilisce i meccanismi di trasporto, gli indirizzi e le porte. Il server, ricevuta la richiesta, risponde al client con OK, con la sessione che transita nello stato Ready.

Il client, allora, per avviare il flusso multimediale, invia il comando PLAY, a cui il server risponde con OK e la sessione entra nello stato di Playing.

In un certo momento, il client invia il comando PAUSE per indicare al server di interrompere la riproduzione del flusso, che risponde con OK, confermando che la trasmissione è stata sospesa. Successivamente, il client invia il comando di PLAY, che il server accetta inviando come risposta OK, con la sessione che ritorna nello stato di Playing.

Infine, il client invia il comando TEARDOWN, riferendo al server di rilasciare le risorse che stava utilizzando durante il flusso. A tale comando, il server risponde con OK, con la sessione che torna allo stato di Init.

La possibilità che avvenga una sessione è permessa dai requisiti di sicurezza che vigilano sul Real Time Streaming Protocol, i quali verranno trattati nel capitolo successivo.

Capitolo 4

Sicurezza nel protocollo RTSP

Nei capitoli precedenti sono stati trattati l'architettura e i metodi del protocollo RTSP. Il capitolo corrente, invece, si vuole soffermare sulla sicurezza del protocollo utilizzato per lo streaming.

4.1 Autenticazione in HTTP

Come si è già visto in 2.6, il protocollo RTSP implementa i meccanismi di sicurezza di HTTP, come ad esempio l'autenticazione e la crittografia.

In generale, l'autenticazione è il processo che permette di confermare l'identità di un utente, verificando che lui sia chi dichiara di essere.

Come riferimento dagli autori di RFC 2326 [1], RTSP utilizza Autenticazione Basic e Autenticazione Digest, trattate nell'RFC 2617 [10].

4.1.1 Schema Basic

Il modello Basic si basa sul principio per cui il client deve procedere con l'autenticazione utilizzando un ID-utente e con una password per ogni realm. Un realm è un valore che definisce lo spazio di protezione (protection space), insieme all'URL della risorsa, ed è composto come una stringa opaca, ovvero la sua struttura interna non deve essere interpretata da chi la trasmette o la riceve. Dunque, tale stringa opaca ha la funzionalità di mantenere flessibilità ed integrità, senza richiedere al client di dover comprendere la logica di quei dati. Riguardo il valore del realm, questo può essere confrontato per verificare l'uguaglianza con gli altri domini, ma solo se essi appartengono allo stesso

server. Successivamente, il server provvede alla gestione della richiesta solo se riesce nell'intento di convalidare l'ID utente e la password per lo spazio di protezione dell'URI della richiesta.

L'autenticazione Basic si basa su un meccanismo di sfida e risposta (challenge-response), in cui un client che desidera accedere ad una risorsa protetta non fornisce credenziali valide.

Nella fase iniziale del meccanismo il client invia una richiesta non autorizzata al server per una risorsa all'interno del suo spazio di protezione, questo per segnalare che l'accesso è stato negato, poiché è priva di autenticazione deve rispondere con il codice di stato 401 (Unauthorized). La risposta del server deve, inoltre, includere il campo di intestazione WWW-Authenticate, specificando al client quale schema di autenticazione sia richiesta per accedere alla risorsa.

Il client, dopo aver ricevuto la sfida, deve elaborare la risposta scegliendo lo schema di autenticazione più forte, tra quelli che sono stati proposti dal server, tipicamente Basic o Digest. Il client, se dovesse scegliere di rispondere alla sfida Basic, invia una nuova richiesta identica a quella precedente, includendo l'intestazione Authorization. Tale intestazione contiene le credenziali che il client ha fornito con il formato: credenziali = <ID-utente:password>, ovvero con la codifica della coppia di informazioni, separata dai due punti (:). Successivamente, se le credenziali sono accettate dal server, questo le soddisfa, altrimenti, restituisce nuovamente un altro codice di errore 401 congiunto ad una nuova sfida.

L'autenticazione Basic è ritenuta debole, secondo gli standard di crittografia moderna, poiché permette la trasmissione in chiaro, attraverso internet, della password dell'utente. Per questa vulnerabilità, tale autenticazione può provocare all'utente di essere attaccato con grande facilità dai malintenzionati. Una grave minaccia può essere l'intercettazione della password da parte di un attaccante in ascolto, che può così accedere a risorse o servizi riservati all'utente. Un altro rischio è lo Spoofing, in cui un attaccante può trarre in inganno il malcapitato, inducendo l'utente a connettersi ad un server fittizio che richiede l'autenticazione Basic. In questo modo il malintenzionato può recuperare la password dell'utente, senza quest'ultimo ne abbia la conoscenza, infatti può

lanciare un errore per nascondere l'attacco. Infine, un'altra pratica può essere quella di sfruttare un proxy compromesso per intercettare la comunicazione. Un attacco ad un server proxy può preoccupare poiché questi sono molto ricorrenti e svolgono funzioni essenziali per la sicurezza, l'efficienza e la traduzione di indirizzi e protocolli.

Considerando tali limiti e vulnerabilità dell'autenticazione Basic, è sicuramente evitabile la sua implementazione per proteggere dati sensibili, la cui riservatezza va protetta. RTSP, comunque, permette lo schema Basic, se questo utilizza una connessione Transport Layer Security (TLS) [11], che cifra il canale di comunicazione sottostante. TLS agisce sui protocolli che sono orientati alla connessione, come TCP, creando un tunnel sicuro e dotato di cifratura tra un client e un server, riuscendo a garantire riservatezza.

4.1.2 Schema Digest

Lo schema Digest nasce come framework di autenticazione di HTTP per risolvere le vulnerabilità affrontate precedentemente con il modello Basic, mantenendosi, comunque, simile in alcuni aspetti. Per questioni di sicurezza, RTSP richiede in modo esplicito l'implementazione Digest, preferendola all'autenticazione Basic.

Lo schema Digest è concettualmente simile a Basic, infatti è basato su un modello di sfida e risposta, con l'aggiunta di codifiche crittografiche, che lo rendono più sicuro. L'autenticazione Digest, al contrario di Basic, assicura che la trasmissione della password dell'utente, come credenziale, non avvenga in chiaro.

La cifratura della chiave avviene grazie ad una combinazione della password (messaggio in chiaro) con una chiave, ovvero una sequenza di bit che serve per la crittografia. Nello specifico, il server genera un valore nonce, ovvero un numero casuale che viene adoperato una sola volta per la sfida. Il valore nonce viene utilizzato da un algoritmo di crittografia, solitamente con Message Digest 5 (MD-5), con lo scopo di convertire la password in un formato che può risultare illeggibile ad un malintenzionato che tenta di intercettare la comunicazione.

MD5 viene definito in RFC 1321 [12] come una funzione hash crittografica, ovvero un algoritmo che mappa i dati e accetta un messaggio di lunghezza

variabile producendone uno di lunghezza fissa. Riflettendosi sull'autenticazione Digest, usa la crittografia prendendo in input la password in chiaro e restituendo come output il messaggio cifrato. Il protocollo MD5 in generale è considerato un algoritmo di base, ma è utile combinato con Digest, infatti quest'ultimo fornisce elementi per mitigare alcuni rischi. Digest introduce:

- Nonce del Server (nonce): viene generato dal server per ogni sfida e può includere il time-stamp per limitare l'uso in un arco temporale prestabilito. Nonce ha l'utilità di rendere un attacco di riproduzione molto più difficile da compiere.
- Nonce del Client (cnonce): è scelto dal client e permette di difendersi da un attacco con testo in chiaro scelto (Chosen-plaintext attack), infatti l'utente può variare ogni volta l'input dell'hash.
- Contatore del Nonce (nc): è un contatore che indica il numero di richieste che sono state inviate da un client che usa il nonce corrente. Nc permette al server di accorgersi di richieste fittizie quando il valore viene visto due volte.

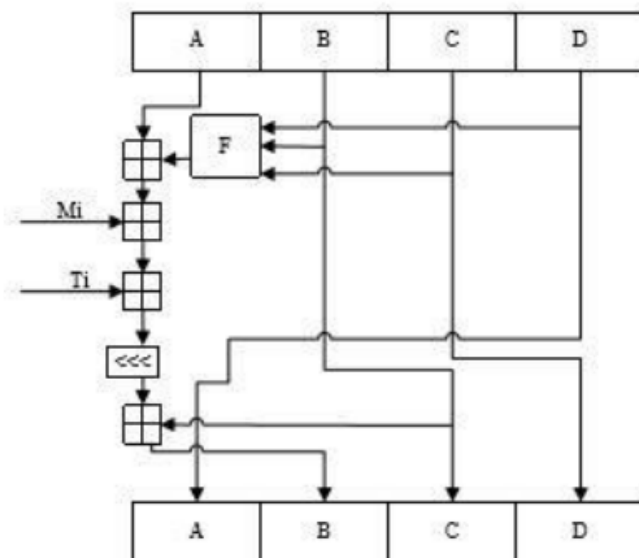


Figura 4.1: Funzionamento dell'algoritmo MD5 [13]

L'immagine mostrata nella Figura 4.1 descrive in modo semplice cosa accade a ogni singolo passo dell'algoritmo MD5. L'algoritmo si avvale di quattro

registri A, B, C, D di 32 bit che servono per memorizzare lo stato interno. Il processo comincia dal registro A, viene recuperato il valore che possiede all'interno e si somma con il risultato di una funzione non lineare F, che prende in input i valori dei registri B, C, D. Tale somma è solo parziale, infatti vanno aggiunti anche M_i e T_i , che sono rispettivamente la parola i -esima del blocco del messaggio iniziale che deve essere cifrato e la costante specifica per ogni passo. Successivamente, il risultato di tale somma subisce uno shift circolare a sinistra ($\ggg$) di un numero di bit predefinito per ogni passo. Infine, al risultato viene aggiunto il valore originario presente nel registro B che viene sovrascritto dalla nuova somma. Inoltre, si può notare un mescolamento per gli altri registri (D viene assegnato ad A, B a C e C a D). Dunque, tale immagine mostra come l'algoritmo mescola le sequenze di bit della chiave, con il passo che viene ripetuto 16 volte per ogni blocco di messaggio.

L'algoritmo MD5 è considerato obsoleto e debole per gli standard di sicurezza, infatti è suscettibile di calcoli di predizione del Digest, inoltre non si preoccupa di cifrare contenuto dei messaggi.

Ritornando allo schema Digest, la Challenge-Response è strutturata nello stesso modo dell'autenticazione Basic, con la differenza che il server include il parametro nonce nell'header della risposta WWW-Authenticate, che ha la funzione di assicurare che la cifratura della chiave sia specifica per quella sessione, evitando possibili attacchi di riproduzione. Per permettere l'autenticazione Digest, è richiesto al client di includere nella sua richiesta un contatore del nonce, che deve essere incrementato a ogni invio e assicura che l'uso di quel valore sia unico. Attraverso tale pratica, l'autenticazione Digest viene considerata più sicura e superiore alla Basic.

Tuttavia, lo schema Digest, nonostante il vantaggio rispetto al modello Basic, presenta dei limiti che lo rendono meno efficiente. In primo luogo, come riferito dagli autori in [10], lo schema Digest non risponde in modo completo alle esigenze di sicurezza. Il protocollo Digest non prevede la crittografia del contenuto dei messaggi, ma si limita a risolvere solamente falle e rischi dell'autenticazione di base. Inoltre, è un meccanismo ritenuto debole dal punto di vista crittografico, infatti è soggetto a potenziali attacchi, come quelli a forza bruta o a man-in-the-middle (MitM). Infine, l'algoritmo che utilizza (MD5),

in tempi più recenti, è stato considerato obsoleto e inadeguato per alcune sue debolezze crittografiche.

4.1.3 Implementazione in RTSP

L'autenticazione in Real Time Streaming Protocol ha una notevole importanza per la sicurezza per il protocollo, utilizzando i meccanismi di controllo adoperati da HTTP. Tale conformità permette a RTSP di poter utilizzare dei meccanismi già collaudati, senza dover inventare nuove pratiche di sicurezza. Il protocollo RTSP utilizza gli schemi di autenticazione Basic e Digest, i quali devono essere implementati sia dai client, che devono accedere a risorse che devono essere protette, sia dai server.

In generale, adotta il modello di autenticazione sfida-risposta (challenge-response), il quale come già visto precedentemente in 4.1.1, viene utilizzato da HTTP. Inoltre, RTSP riprende gli header e codici di stato per comunicare durante la sfida tra client e server.

In tale sfida, il client invia una richiesta priva di credenziali, a cui il server risponde utilizzando l'header WWW-Authenticate, insieme al codice di stato 401 (Unauthorized), e richiedendo le informazioni dell'utente, ovvero l'user-id e la password.

Quando RTSP adotta l'autenticazione Basic, tenta di mitigare i suoi problemi di vulnerabilità, infatti tale schema non garantisce la sicurezza, poiché trasmette la password in chiaro. Per una migliore sicurezza, il protocollo di Streaming, utilizza Transport Layer Security, che crea un canale crittografico con lo scopo di proteggere la confidenzialità. Per le sessioni RTSP che richiedono protezione a livello di trasporto e sono veicolate su connessioni TCP, l'utilizzo di TLS è un requisito obbligatorio. Invece, l'implementazione dell'autenticazione Digest da parte di RTSP 2.0, è praticamente identica a quella di HTTP, con lo stesso utilizzo dei nonce.

RTSP, oltre a utilizzare gli schemi di HTTP, introduce una nuova gestione dell'autenticazione che è basata sui server proxy, i quali servono per veicolare il traffico dei dati. RTSP distingue:

- Autenticazione End-to-End: tra il client e il server di origine che ospita la risorsa desiderata.
- Autenticazione Proxy: vi è un proxy che richiede la verifica al client e dopo l'inoltra alla destinazione finale.

L'autenticazione end-to-end utilizza gli header WWW-Authenticate, Authorization e Authentication-Info che i server proxy non devono modificare quando li intercettano. Essi devono essere inoltrati mantenendoli identici a come sono stati ricevuti. Infatti, la funzione di proxy è unicamente quella di garantire l'autenticazione tra il client e il server d'origine della risorsa, con il meccanismo che deve essere indipendente dalla presenza di intermediari.

L'autenticazione proxy funziona in modo abbastanza diverso, non è il server di origine a richiedere la verifica, ma è un intermediario che sollecita l'identificazione del client, prima che tale richiesta sia inviata alla destinazione finale. Tale autenticazione adopera degli header diversi:

- Proxy-Authenticate: viene utilizzato dal proxy in seguito a una challenge di autenticazione, con il server che invia la risposta 407 Proxy Authentication Required, sfidando un client o un intermediario precedente.
- Proxy-Authorization: contiene le credenziali che sono state fornite da un proxy o da un client in seguito alla ricezione di una sfida. Quando la sfida viene accettata, permette loro di identificarsi a chi richiede l'autenticazione.
- Proxy-Authentication-Info: è utilizzato dai proxy per riferire informazioni dopo che un'autenticazione è stata completata con successo.

In generale, gli header dell'autenticazione proxy si definiscono hop-by-hop, ovvero composti da hop, cioè passi attraverso la catena di comunicazione composta da proxy che si instaura tra il client e il server d'origine.

4.2 Sicurezza

La sicurezza dei dati in RTSP richiede particolare attenzione, infatti bisogna garantire riservatezza e integrità sui due canali di comunicazione, ovvero di flusso e di controllo, gestiti rispettivamente da RTP e RTSP.

4.2.1 Canale di controllo

La sicurezza sul canale di controllo RTSP è assicurata dall'implementazione di Transport Layer Security, che si concentra sui comandi che vengono inviati durante la sessione, come SETUP, PLAY, PAUSE e TEARDOWN. Quando TLS gestisce RTSP, si usa l'identificatore URI rtsp, al posto di rtsp, soffermando l'attenzione sull'ultimo carattere della stringa, ovvero s, che indica sicure.

Nel caso in cui un client tenti di stabilire una connessione su un canale non sicuro, quindi con URI rtsp, quando la politica del server richiede sicurezza per una certa risorsa sensibile, tale azione non è direttamente eseguibile. Infatti, il server risponde alla richiesta non sicura con il codice di stato 301 Redirect, insieme alla corretta URI di Location, che utilizza lo schema rtsp, chiedendo, dunque, al client di indirizzare la sua richiesta verso un'altra ubicazione. In generale, TLS fornisce non solo la confidenzialità del suo trasporto, attraverso la crittografia dei messaggi RTSP di controllo, ma permette anche l'autenticazione del server.

4.2.2 Canale di flusso

Per quanto riguarda la sicurezza del canale del flusso multimediale, bisogna soffermarsi sui protocolli di consegna, infatti RTSP non si preoccupa direttamente della trasmissione dei dati multimediali. Tali dati possono essere sensibili e soggetti da intercettazioni da parte di malintenzionati che si mettono in ascolto su quel canale, quindi c'è bisogno che il protocollo di consegna scelto nella sessione fornisca un modo per garantire un adeguato livello di confidenzialità per riservare la sicurezza. RTSP risolve la situazione di confidenzialità incerta attraverso due pratiche: l'utilizzo di Secure RTP (SRTP) e con la gestione delle chiavi crittografiche attraverso MIKEY.

Con l'adozione di SRTP per la salvaguardia del flusso multimediale, i client e i server che utilizzano RTP devono implementare Secure RTP, con l'aggiunta del supporto del profilo SAVP (Secure RTP Profile for Audio and Video Conferences). Il protocollo di sicurezza, oltre a fornire confidenzialità al canale, assicura anche l'integrità del messaggio e l'autenticazione del server di origine attraverso dei meccanismi che prevengono attacchi di replay, manipolazione dei dati o rallentamenti del traffico. SRTP nella sua implementazione adopera due algoritmi: AES-CM e HMAC-SHA1, rispettivamente per la cifratura e per l'integrità.

4.2.3 AES

In particolare, volendo soffermarsi su uno degli algoritmi, ovvero AES-CM (AES-Counter Mode), questo è conosciuto come una variante dell'AES (Advanced Encryption Standard). Nella Figura 4.2 viene mostrato un AES-128 base, che è un cifrario a blocchi a 128 bit, dotato di una struttura caratterizzata da una serie di round. Tale algoritmo di crittografia è uno dei più utilizzati e sicuri, con la funzionalità di proteggere le informazioni sensibili.

Si parte dal plaintext, che è il messaggio in chiaro che va cifrato, attraverso dei blocchi di 128 bit. A sinistra, invece, si può notare la chiave segreta composta da 128 bit, che viene espansa per creare le round keys. Nella prima fase dell'algoritmo il messaggio in chiaro viene combinato con la prima chiave, utilizzando l'operazione di XOR, con il risultato che crea il blocco State. Successivamente, lo State esegue 10 round sempre con la stessa operazione di xor con le varie chiavi.

Ogni round è composto da quattro operazioni:

- SubBytes compie delle sostituzioni in base ad una tabella predefinita.
- ShiftRows effettua un shift ciclico verso sinistra delle righe dello stato.
- MixColumns combina i byte all'interno di ogni colonna.
- AddRoundKey applica allo stato un'operazione di xor con la round key corrente.

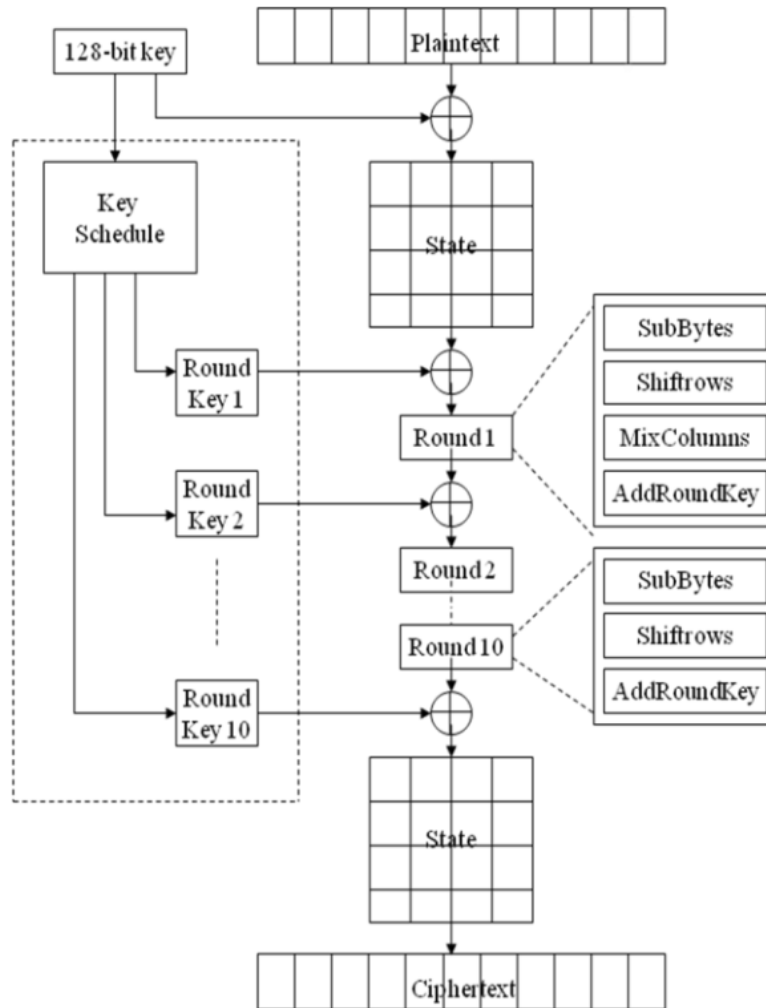


Figura 4.2: Struttura dell'algoritmo AES [14]

4.2.4 MIKEY

Un'altra soluzione di RTSP per risolvere il problema della sicurezza può essere l'utilizzo del meccanismo MIKEY, che si adopera per la gestione delle chiavi e richiede l'implementazione del Multimedia Internet KEYing, che è stato definito in RFC 3830 [15].

MIKEY ha il compito di facilitare l'implementazione di un stato di sicurezza in modo efficiente. Infatti, fornisce una soluzione per la gestione delle chiavi per supportare i protocolli di sicurezza, in particolare per SRTP. Quando un client RTSP implementa SRTP, deve essere utilizzato anche il meccanismo di gestione delle chiavi MIKEY.

Durante la creazione della sessione RTSP, il client, prima di inviare il comando di SETUP al server, genera un messaggio MIKEY che viene codificato e deve

essere incluso nella richiesta come parametro. Quando il server riceve la richiesta di SETUP, genera un messaggio di risposta MIKEY, codificato e incluso nella risposta del comando che viene trasmesso al client.

Nel caso in cui si dovesse verificare un errore durante l'elaborazione di MIKEY, il server deve utilizzare nella risposta al client il codice 466 (Key Management Error). Un altro codice di errore che compare spesso è 403 (Forbidden), il quale deve essere inviato dal server quando si usa un trasporto SRTP, con il client che non deve includere il parametro MIKEY.

In generale, MIKEY offre vantaggi come la riservatezza grazie alla sua gestione delle chiavi, ma anche velocità, infatti supporta il pipelining delle richieste RTSP, senza dover seguire necessariamente una sequenza fissa.

4.2.5 Crittografia in presenza di proxy

Quando si inserisce un proxy all'interno di una comunicazione crittografata che utilizza Transport Layer Security, si potrebbe compromettere in modo notevole il livello di sicurezza lungo il canale. RTSP, per mantenere alta la sicurezza durante la comunicazione aderisce ai seguenti principi:

- Proxy non trasparenti: si usano solo quelli visibili e riconosciuti sia dal client che dal server.
- Fiducia nei confronti dei proxy: client e server devono riporre fiducia nei proxy intermediari, in base alle informazioni fornite durante l'utilizzo del protocollo TLS.
- Autenticazione Hop-by-Hop: quando si applica TLS, il client non eseguirà l'autenticazione con il server originario, ma con il proxy che agisce da intermediario.

RTSP introduce dei meccanismi per consentire l'utilizzo dei server proxy, mantenendo comunque un controllo elevato della sicurezza. Tali meccanismi servono per la gestione delle credenziali e per l'approvazione della catena di autenticazione, questi sono veicolati rispettivamente negli header Accept-Credentials e Connection-Credentials.

L'header Accept-Credentials viene utilizzato dai client verso i proxy intermedi per specificare le politiche di autorizzazione da applicare per la gestione dei certificati TLS del next hop, ovvero il server proxy successivo nella catena. Vengono seguite tre politiche di autorizzazione:

- Any: il proxy che è obbligato ad accettare qualsiasi certificato che gli è stato presentato, ma tale politica non è raccomandata, poiché può essere pericolosa e per nulla sicura.
- Proxy: il proxy deve seguire le proprie politiche interne per accettare il certificato.
- User: il proxy inoltra al client le credenziali del server successivo nella catena.

Nel caso in cui il client ometta l'header Accept-Credentials nella sua richiesta, allora nella catena si applica la politica Proxy.

Un altro header utilizzato da RTSP è Connection-Credentials, il quale viene utilizzato lungo la catena di credenziali per il next hop che richiede l'approvazione. Si usa quando un client invia una richiesta che il proxy non può approvare automaticamente e, dunque, lo invita a rispondere con l'header Connection-Credentials. Si stabilisce una procedura TLS di approvazione:

1. Il server proxy prova a connettersi con il next hop.
2. Il proxy verifica le credenziali del next hop, in particolare se il codice hash del suo certificato sia presente nell'header Accept-Credentials.
3. Il proxy verifica l'hash, se non corrisponde o non è presente, interrompe la catena di comunicazione per la richiesta RTSP, risponde con il codice di errore 470 (Connection Authorization Required).
4. Nel caso in cui l'hash non venga verificato, allora il proxy nell'header Connection-Credentials inserisce l'identità e il certificato del next hop.
5. Il client decide se accettare il certificato del proxy che ha ricevuto. In questo caso deve inviare nuovamente la richiesta originale, ma modificando l'header Accept-Credentials con l'inclusione dell'hash del certificato che ha appena approvato.

6. Il proxy inoltra la richiesta che è stata autorizzata al next hop della catena RTSP.

Questa procedura provoca un notevole ritardo sul primo messaggio inviato dal client, infatti aumenta il Round Trip Time (RTT), ovvero il tempo che intercorre tra l'invio di un segnale e la ricezione della conferma del suo arrivo. In compenso, la procedura consente a un client di utilizzare un modello di fiducia lungo la catena di comunicazione, con l'approvazione dei proxy che avviene in modo dinamico.

Dopo aver trattato il protocollo RTSP dal punto di vista teorico, nel capitolo successivo verrà mostrata un caso d'uso pratico.

Capitolo 5

Realizzazione di un Server RTSP

5.1 Contesto dello Streaming Multimediale

Nei capitoli precedenti si è affrontato lo studio di Real Time Streaming Protocol, in cui ci si è soffermati sulle sue caratteristiche e sulle sue funzionalità. L'obiettivo di questo capitolo, invece, è descrivere la realizzazione di un sistema di streaming end-to-end, illustrandone l'architettura e i componenti utilizzati. Negli ultimi decenni lo streaming multimediale ha goduto di una crescita esponenziale, soprattutto per quanto riguarda la trasmissione dei dati in tempo reale.

Il Real Time Streaming Protocol, come già visto precedentemente, è un protocollo di livello applicativo che fornisce un framework per il controllo e la consegna on-demand di audio e video [1].

5.1.1 Supporto di RTSP nelle applicazioni di streaming multimediale

RTSP supporta diverse applicazioni fondamentali nello streaming multimediale come:

- Riproduzione di contenuti on-demand: con il protocollo che fornisce dei meccanismi per poter riprodurre su richiesta clip o dati archiviati. RTSP, per tale funzionalità, consente a un client di avviare, mettere in pausa e

interrompere definitivamente la trasmissione del contenuto che desidera. Inoltre, è possibile compiere una ricerca casuale su un qualsiasi punto della timeline, vale a dire che permette a un client di poter scegliere il punto della riproduzione da cui iniziare o riprendere la fruizione del contenuto che aveva richiesto.

- Videosorveglianza con registrazione: con RTSP che offre al client due servizi. Il client non solo riceve il flusso in tempo reale, ma può anche riprodurre in modo continuo un'altra sezione che non corrisponda al momento live. Infatti, Real Time Streaming Protocol permette agli utenti di usufruire di un flusso live, che contemporaneamente viene archiviato e conservato dal server, che è pronto ad offrirlo al client su richiesta.
- Live broadcast: con il server RTSP che ricopre il ruolo di nodo di distribuzione ed invia lo stesso contenuto, tramite unicast, a più client che usufruiscono di tale servizio contemporaneamente. Per via di questa caratteristica, il client può accedere a un contenuto che viene generato allo stesso tempo di quando viene consegnato.
- Sessioni Multiparty: con RTSP che supporta l'invito di un server media in una conferenza o in una sessione preesistente. Tale modalità fornita dal protocollo è spesso utilizzata per le applicazioni di insegnamento distribuite.

5.1.2 Streaming on-demand

Nel corso della trattazione di RTSP si è parlato di streaming on-demand, ovvero riproduzioni di contenuti che sono già stati archiviati e sono pronti per essere trasmessi. Relativamente a questa modalità di streaming, la dipendenza temporale tra la sorgente e la destinazione del contenuto multimediale è meno stringente. Infatti il principio primario da seguire è garantire una riproduzione stabile dopo che è stato avviato il flusso.

Riguardo i media on-demand, di questi si conoscono le durate della riproduzione, che sono fisse e, inoltre, si conoscono al momento della creazione della sessione. I client che attingono agli streaming on-demand possono utilizzare degli

indicatori temporali come Normal Play Time (NPT) per indicare la posizione della timeline della riproduzione da cui vogliono partire, quindi supportando l'accesso in modalità casuale e non solo sequenziale.

5.1.3 Streaming Real Time

La categoria di streaming real time include contenuti come live broadcast o teleconferenze, con la relazione temporale tra sorgente e destinazione che è molto rigorosa rispetto ai flussi on-demand.

In questa categoria è essenziale che il sistema goda di bassa latenza, ovvero che il ritardo che si verifica tra un'azione e la sua reazione sia minima. A differenza dello streaming on-demand, il contenuto usufruibile dal client diventa disponibile con il passare del tempo, la cui durata della riproduzione è sconosciuta.

Un'altra differenza tra le due modalità di streaming è che la riproduzione Real Time spesso non permette di cercare sezioni del flusso. Infatti il server potrebbe avere solamente il compito di inviare il flusso che proviene dalla sorgente, senza che questo venga archiviato per essere riprodotto successivamente in seguito a una richiesta di un client.

In sostanza, il supporto da parte di RTSP per le due modalità di streaming, rende il protocollo flessibile e versatile nel controllare sessioni di riproduzione.

5.2 Architettura delle componenti

La funzione specifica del sistema RTSP che si vuole realizzare in questo capitolo è quella di catturare il flusso video proveniente da una sorgente locale, ovvero la webcam. Con lo scopo, successivamente, di elaborare e rendere disponibile il flusso attraverso un server RTSP, permettendo a un client di usufruire del servizio.

Questa realizzazione permette di analizzare le tre fasi fondamentali dello streaming real time:

- L'acquisizione del flusso grezzo, che in questo caso proviene dalla webcam.

- L'elaborazione del flusso che è troppo pesante per essere veicolato, allora viene compresso in un formato efficiente per essere trasmesso sulla rete.
- La distribuzione del servizio che avviene con l'utilizzo di un server che gestisca il canale di controllo (RTSP) e del flusso (RTP).

5.2.1 Acquisizione del flusso video

L'acquisizione del flusso rappresenta la prima fase dello streaming real time ed è necessario in primo luogo identificare il nome del dispositivo video da utilizzare. In particolare, in questa realizzazione la sorgente dei dati è costituita dalla webcam HP TrueVision HD Camera che è integrata nel computer. I sistemi Windows per gestire i loro dispositivi multimediali utilizzano DirectShow, ovvero un framework che fornisce un'interfaccia standardizzata per l'accesso alle periferiche.

Per elencare i dispositivi di input DirectShow (dshow), come riferito in [16], è necessario eseguire un'operazione particolare di FFmpeg nel prompt dei comandi. Quest'ultimo è un'interfaccia testuale che consente di eseguire i comandi da tastiera. L'operazione da eseguire è:

```
ffmpeg -list_devices true -f dshow -i dummy
```

In questo comando distinguiamo:

- ffmpeg: è un convertitore multimediale universale che può leggere un'ampia varietà di input, inclusi dispositivi di acquisizione/registrazione in tempo reale, filtrarli e transcodificarli in una miriade di formati di output [17].
- -list_devices true: fornisce un elenco di tutti i dispositivi di input disponibili, mostrando i loro nomi e gli indici. [16]
- -f dshow: specifica che il formato dei dispositivi in input deve essere dshow (DirectShow).
- -i dummy: è un input fittizio inserito solo per attivare la scansione dei dispositivi del sistema.

Dopo aver eseguito l'operazione nel prompt dei comandi verrà mostrato in output, come in Figura 5.1, il nome del dispositivo, in questo caso la webcam HP TrueVision HD Camera.

```
C:\Users\franc\ffmpeg\bin>ffmpeg -list_devices true -f dshow -i dummy
ffmpeg version 8.0-essentials_build-www.gyan.dev Copyright (c) 2000-2025 the FFmpeg developers
  built with gcc 15.2.0 (Rev8, Built by MSYS2 project)
  configuration: --enable-gpl --enable-version3 --enable-static --disable-w32threads --disable-autodetect --en
  able-fontconfig --enable-iconv --enable-gnutls --enable-libxml2 --enable-gmp --enable-bzlib --enable-lzma --en
  able-zlib --enable-lbsrt --enable-libssh --enable-libzmq --enable-avisynth --enable-sdl2 --enable-libwebp --e
  nable-liby264 --enable-libx265 --enable-libxvid --enable-libaom --enable-libopenjpeg --enable-libvpx --enable-
  mediafoundation --enable-libass --enable-libfreetype --enable-libfribidi --enable-libharfbuzz --enable-libvids
  tab --enable-libvmaf --enable-libzimg --enable-amf --enable-cuda-llvm --enable-cuvid --enable-dxva2 --enable-d
  3d11va --enable-d3d12va --enable-ffnvcodec --enable-libvpl --enable-nvdec --enable-nvenc --enable-vaapi --enab
  le-opengl --enable-libgme --enable-libopenmpt --enable-libopencore-amrwb --enable-libmp3lame --enable-libtheor
  a --enable-libvo-amrwbenc --enable-libgsm --enable-libopencore-amrnb --enable-libopus --enable-lspspeex --enab
  le-libvorbis --enable-librubberband
  libavutil      60. 8.100 / 60. 8.100
  libavcodec     62.11.100 / 62.11.100
  libavformat    62. 3.100 / 62. 3.100
  libavdevice    62. 1.100 / 62. 1.100
  libavfilter    11. 4.100 / 11. 4.100
  libswscale     9. 1.100 / 9. 1.100
  libswresample  6. 1.100 / 6. 1.100
[dshow @ 0000027d262737c0] "HP TrueVision HD Camera" (video)
[dshow @ 0000027d262737c0] Alternative name "@device_pnp\{?usb\vid_04f2&pid_b75e&mi_00#6&cabd04b&0&0000#{6
5e8773d-8f56-11d0-a3b9-00a0c9223196}\global"
[dshow @ 0000027d262737c0] "Headset Microphone (Realtek(R) Audio)" (audio)
[dshow @ 0000027d262737c0] Alternative name "@device_cm_{33D9A762-90C8-11D0-BD43-00A0C911CE86}\wave_{73D0032
3-614D-4586-8244-6F012F6666FA}"
[dshow @ 0000027d262737c0] "Gruppo microfoni (Tecnologia Intel® Smart Sound per microfoni digitali)" (audio)
[dshow @ 0000027d262737c0] Alternative name "@device_cm_{33D9A762-90C8-11D0-BD43-00A0C911CE86}\wave_{AFA7D02
4-07F8-4D6D-BC66-08D1808F8E63}"
error opening input file dummy.
```

Figura 5.1: Lista dei dispositivi video e audio riconosciuti da FFmpeg con il modulo DirectShow

Una volta acquisito il nome del dispositivo, è necessario utilizzare FFmpeg per catturare il flusso proveniente dalla webcam.

5.2.2 L'elaborazione e la compressione del flusso

Il flusso video che è stato acquisito presenta delle dimensioni abbastanza elevate e lo rendono inadatto a essere trasmesso in tempo reale in rete.

Secondo quanto riportato dal framework go2rtc [18], per la compressione del flusso è consigliabile adottare lo standard di codifica video H.264, che viene utilizzato nelle applicazioni di streaming ad alte prestazioni (High Performance Streaming). Come è evidenziato anche in [19], lo standard H.264 è una delle modalità di compressione più efficienti e diffuse attualmente. Gli standard per la compressione delle informazioni audiovisive, infatti facilitano lo scambio di dati a livello globale sotto forma di video digitale.

Per ottenere il flusso RTSP da una qualsiasi webcam si utilizza la suite FFmpeg. [18]. Tale operazione viene eseguita in un file in formato YAML, utilizzato per la serializzazione dei dati e si spesso nei file di configurazione. Il comando che va eseguito nel file YAML, come illustrato nella Figura 5.2, è:

```
streams:
```

webcam:

- "ffmpeg:device?video=<NOME_WEBCAM>#video=h264"

In particolare:

- ffmpeg: indica al sistema che il flusso deve essere gestito tramite il framework FFmpeg.
- device: specifica che l'origine del flusso è un dispositivo locale.
- ?: si usa per introdurre i parametri di configurazione.
- video =<NOME_WEBCAM>: indica che il parametro riguarda la sorgente video, specificando il nome del dispositivo di origine del flusso.
- #: introduce un commento con i vari parametri.
- video=h264: specifica che il video deve essere codificato in H.264.

Il file YAML non è ancora completo, infatti, la sua configurazione finale sarà mostrata nel paragrafo successivo.

5.2.3 Distribuzione del servizio RTSP

Per poter fornire un servizio RTSP accessibile al client è stato utilizzato il framework go2rtc. Si tratta di un'applicazione progettata per lo streaming delle telecamere con supporto RTSP, che fornisce un server open-source, sviluppato da AlexxIT [18], e scritto nel linguaggio di programmazione Go.

go2rtc, all'interno del progetto, riceve lo stream elaborato da FFmpeg e lo distribuisce tramite Real Time Streaming Protocol.

Per completare la configurazione del file YAML, si possono seguire due strategie differenti:

1. Integrando FFmpeg nelle variabili d'ambiente.
2. Definendo in modo esplicito nel file YAML il percorso di FFmpeg. Questa è la strategia adottata nel progetto, pertanto è necessario aggiungere nel file di configurazione (Figura 5.2):

```
ffmpeg:
  bin: "C:/Users/<NOME_UTENTE>/ffmpeg/bin/ffmpeg.exe"
```

```
ffmpeg:
  bin: "C:/Users/franc/ffmpeg/bin/ffmpeg.exe"

streams:
  webcam:
    - "ffmpeg:device?video=HP TrueVision HD Camera#video=h264"
```

Figura 5.2: Configurazione del file YAML

Una volta completata la configurazione del file YAML, è possibile procedere alla fase successiva, ovvero l'avvio del server go2rtc tramite il prompt dei comandi. Il comando specifico da eseguire nella directory dove si trova il file YAML è:

```
C:\go2rtc.exe -config go2rtc.yaml
```

All'esecuzione del comando, go2rtc esegue una sequenza di operazioni di inizializzazione che vengono riportate nell'output. Un esempio tipico è mostrato nella Figura 5.3.

```
C:\Users\franc>C:\go2rtc.exe -config go2rtc.yaml
15:50:30.618 INF go2rtc platform=windows/amd64 revision=be80eb1 version=1.9.11
15:50:30.654 INF config path="C:\\Users\\franc\\go2rtc.yaml"
15:50:30.658 INF [api] listen addr=:1984
15:50:30.658 INF [rtsp] listen addr=:8554
15:50:30.696 INF [webrtc] listen addr=:8555
```

Figura 5.3: Esecuzione del programma go2rtc

Dall'output della Figura 5.3 è possibile osservare la versione di go2rtc in uso e il file di configurazione YAML caricato dalla directory indicata. In particolare, nell'output compare la riga:

```
13:53:49.576 INF [rtsp] listen addr=:8554
```

La riga dell'output indica che il modulo RTSP è in ascolto sulla porta 8554, rendendo il server accessibile ai client che intendono stabilire una sessione tramite il protocollo.

5.2.4 Connessione del client

Successivamente, per connettersi al server RTSP si può stabilire una connessione attraverso VLC Media Player [18], che è un lettore multimediale in grado di riprodurre file audio e video di diversi formati.

Accedendo al lettore VLC bisogna inserire, come mostrato nella Figura ??, un URL di rete con il seguente formato:

Schema : //Indirizzo IP : Porta TCP / Risorsa

Analizzando la riga nel dettaglio, si ha:

- Schema: identifica il protocollo che si sta adoperando, in particolare, in questo caso, è stato inserito "rtsp", per indicare che si tratta del Real Time Streaming Protocol.
- Indirizzo IP: specifica l'identificatore numerico del server a cui il client VLC deve connettersi.
- Porta TCP: indica la porta su cui il processo del server RTSP è in ascolto, in questo caso è stato specificato che si tratta del porta 8554.
- Risorsa: rappresenta la risorsa multimediale richiesta dal client sul server, il cui nome è stato definito nel file YAML, in questo caso è stato inserito webcam.

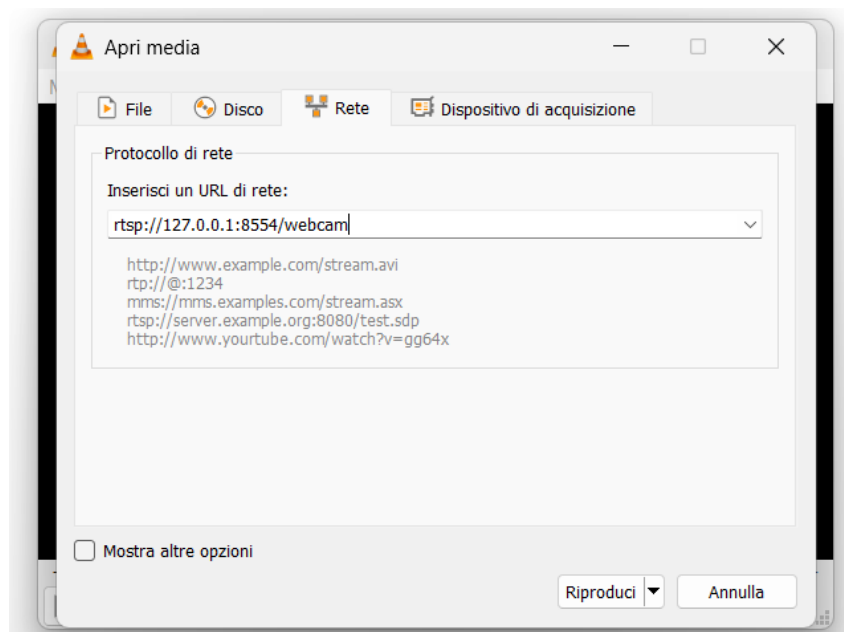


Figura 5.4: Inserimento dell'URL di rete all'interno di VLC

Una volta eseguita l'operazione, con il server attivo, si può riprodurre il flusso con VLC che svolge le operazioni che seguono:

1. Dopo aver completato il SETUP RTSP, riceve i pacchetti RTP sulle porte UDP.
2. Riordina i pacchetti attraverso il numero di sequenza.
3. Ricostruisce i frame completi, con i pacchetti RTP che vengono elaborati.
4. Decodifica i frame passandoli al decoder H.264.
5. Visualizza il video sullo schermo, come mostrato nella Figura 5.5.

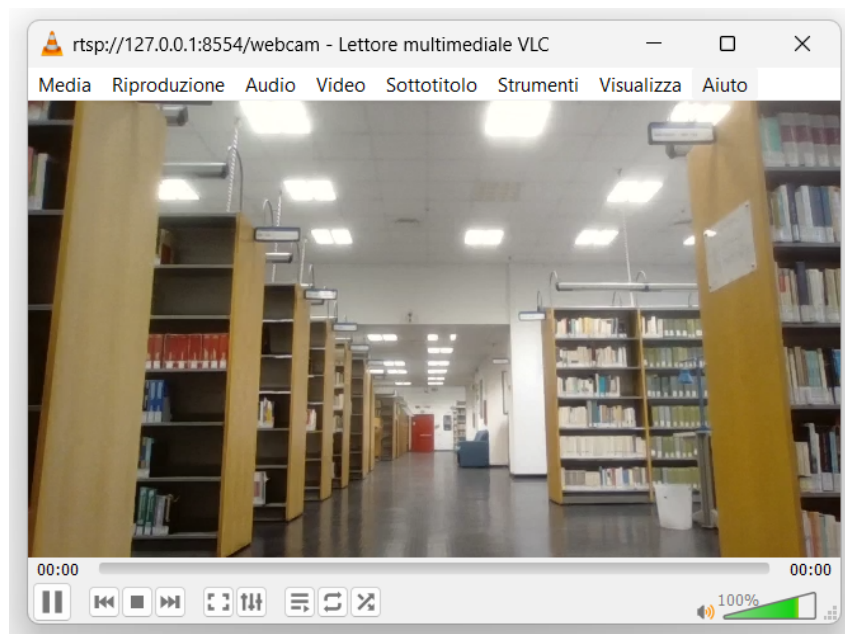


Figura 5.5: Riproduzione del flusso

5.3 Conclusioni

Dunque, si è raggiunto l'obiettivo di creare un sistema di streaming video basato su RTSP, dimostrando la corretta integrazione di tutti i componenti, ovvero:

- L'acquisizione del flusso proveniente dalla webcam.
- L'elaborazione del flusso attraverso il framework FFmpeg.

- La distribuzione del servizio mediante go2rtc come server RTSP.
- La ricezione del flusso da parte del client tramite VLC Media Player.

Il sistema RTSP mostrato nel corso del capitolo rappresenta una soluzione completa e conforme agli standard per la distribuzione di flusso video in tempo reale. Questo progetto costituisce una base solida per lo sviluppo di applicazioni in ambito di videosorveglianza, di live broadcast e di videoconferenza.

Capitolo 6

Conclusioni

L'obiettivo di questa tesi era analizzare in modo approfondito il Real Time Streaming Protocol, evidenziandone le caratteristiche che riguardano l'architettura, i suoi meccanismi di funzionamento e anche le criticità legate alla gestione delle sessioni multimediali. Per osservare il comportamento del protocollo, è stato realizzato un caso d'uso pratico mediante l'integrazione di strumenti come go2rtc, FFmpeg e VLC Media Player.

Il lavoro della tesi è composto da due sezioni principali. Nella prima parte viene condotta un'analisi teorica del protocollo, dove è descritta la sua architettura modulare, l'interazione con altri protocolli e la gestione delle sessioni. Inoltre, in questa sezione è stato approfondito il tema della sicurezza, con particolare attenzione ai meccanismi di autenticazione e alla vulnerabilità dei canali di controllo e di trasporto. Nella seconda parte della tesi, di carattere sperimentale, è stata illustrata la configurazione di un server di streaming.

Lo studio condotto in questo lavoro ha messo in evidenza la flessibilità del protocollo RTSP. Grazie alla sua modularità è utile per applicazioni on-demand o real time, adatte alla videosorveglianza, ai live broadcast e alle videoconferenze. Per sviluppi futuri, il lavoro potrebbe essere esteso in diverse direzioni, ad esempio mediante l'integrazione di sistemi di cifratura o l'impiego di server proxy per ottimizzare la distribuzione dei flussi.

Bibliografia

- [1] H. Schulzrinne, A. Rao e R. Lanphier, *RFC2326: Real Time Streaming Protocol (RTSP)*, USA, 1998.
- [2] H. Schulzrinne, A. Rao, R. Lanphier, M. Westerlund e M. Stiemerling, *Real-Time Streaming Protocol Version 2.0*, RFC 7826, dic. 2016. DOI: 10.17487/RFC7826. indirizzo: <https://www.rfc-editor.org/info/rfc7826>.
- [3] D. Biacco, «Integrazione dei protocolli TCP/IP e ZigBee in un network capable application processor (NCAP) IEEE 1451,»
- [4] L. Jianbing e C. Shuhui, «The Design and Implementation of RTSP/RTP Multimedia Traffic Identification Algorithm,» *Journal of Physics: Conference Series*, vol. 1168, n. 5, p. 052 033, feb. 2019. DOI: 10.1088/1742-6596/1168/5/052033. indirizzo: <https://doi.org/10.1088/1742-6596/1168/5/052033>.
- [5] H. Schulzrinne, S. Casner, R. Frederick e V. Jacobson, *RFC3550: RTP: A Transport Protocol for Real-Time Applications*, USA, 2003.
- [6] A. H. Hussein, M. Abu-Alhaija e K. Nairoukh, «New RTP packet payload shrinking method to enhance bandwidth exploitation over RTP protocol,» *International Journal of Advanced Computer Science and Applications*, vol. 11, n. 8, p. 18, 2020. DOI: 10.14569/IJACSA.2020.0110818. indirizzo: <https://doi.org/10.14569/IJACSA.2020.0110818>.
- [7] M. J. Handley e V. Jacobson, *SDP: Session Description Protocol*, RFC 2327, apr. 1998. DOI: 10.17487/RFC2327. indirizzo: <https://www.rfc-editor.org/info/rfc2327>.

- [8] R. Zoto, «Riconoscimento di dispositivi di protezione individuale in ambito industriale tramite infrastruttura cloud,» tesi di dott., Politecnico di Torino, 2024.
- [9] M. Rupp, D. M. Ries e M. Braun, «Video streaming test bed for UMTS network,» *Jun-2016.[Online]. Available: https://publik.tuwien.ac.at/files/pub-et_*, vol. 1, 1905.
- [10] J. Hostetler, S. Lawrence, P. Leach, A. Luotonen e L. Stewart, «HTTP Authentication: Basic and Digest Access Authentication,» RFC 2617, June, rapp. tecn., 1999.
- [11] E. Rescorla e N. Modadugu, «Datagram transport layer security,» rapp. tecn., 2006.
- [12] R. Rivest, *RFC1321: The MD5 message-digest algorithm*, 1992.
- [13] M. Dinakaran, K. N. Darshan e H. Patel, «Analysis of MD5 authentication in various routing protocols using simulation tools,» *IOP Conference Series: Materials Science and Engineering*, vol. 263, n. 4, p. 042014, nov. 2017. DOI: 10.1088/1757-899X/263/4/042014. indirizzo: <https://doi.org/10.1088/1757-899X/263/4/042014>.
- [14] M. T. Khaing e Z. M. Aye, «Image Encryption Based on AES Stream Cipher in Counter Mode,» tesi di dott., MERAL Portal, 2009.
- [15] J. Arkko, E. Carrara, F. Lindholm, M. Naslund e K. Norrman, *RFC 3830: MIKEY: Multimedia Internet KEYing*, USA, 2004.
- [16] FFmpeg Developers, *Devices — FFmpeg Documentation (Description)*, <https://ffmpeg.org/ffmpeg-devices.html#Description>.
- [17] FFmpeg Developers, *FFmpeg — A complete, cross-platform multimedia framework*, <https://ffmpeg.org/ffmpeg.html>.
- [18] AlexxIT, *go2rtc*, <https://github.com/AlexxIT/go2rtc>, 2025.
- [19] A. Nurrohman e M. Abdurrohman, «High Performance Streaming Based on H264 and Real Time Messaging Protocol (RTMP),» in *2018 6th International Conference on Information and Communication Technology (ICoICT)*, 2018, pp. 174–177. DOI: 10.1109/ICoICT.2018.8528770.